



Università degli Studi di Salerno

Corso di Laurea Triennale in Informatica

Migrazione verso la Crittografia Post-Quantum:

Analisi delle problematiche e sfide nella transizione dagli
algoritmi di cifratura tradizionali

Studente: Adriano De Vita

Matricola: 0512117726

Primo Relatore: Prof. Arcangelo Castiglione

Secondo Relatore: Prof. Alfredo De Santis

Anno Accademico 2025–2026

Abstract

Il quantum computing sta emergendo come una tecnologia rivoluzionaria, capace di risolvere problemi complessi a velocità esponenzialmente superiori rispetto ai più potenti calcolatori tradizionali; tuttavia, le minacce da esso derivanti compromettono le fondamenta matematiche di molti standard crittografici presenti in letteratura, come RSA e la crittografia a curve ellittiche (ECC). La prospettiva dell'avvento di un computer quantistico crittograficamente rilevante (CRQC), unita al rischio di attacchi di tipo 'Store Now, Decrypt Later', rende urgente l'adozione di nuovi paradigmi di sicurezza, in linea con gli standard di Post-Quantum Cryptography (PQC) recentemente finalizzati dal NIST, tra cui il FIPS 203 (ML-KEM)[1]. Il presente elaborato si propone di analizzare le problematiche teoriche e le sfide implementative connesse alla transizione verso sistemi quantum-resistant. Durante l'attività pratica di tirocinio si è concretizzata l'integrazione di un meccanismo di PQC ibrido all'interno di una piattaforma web reale basata su architettura a microservizi. Utilizzando librerie crittografiche e framework, è stato sviluppato un sistema che combina l'attuale robustezza degli algoritmi classici (ECDH) con il nuovo standard PQC (ML-KEM), garantendo la confidenzialità dei dati sensibili (Encryption at Rest). Particolare attenzione è stata posta poi alla gestione della retro-compatibilità, attraverso lo sviluppo di un componente per la migrazione automatica e trasparente degli account legacy e l'adeguamento del client web per la notifica dello stato di sicurezza all'utente. I risultati ottenuti dimostrano quindi la fattibilità tecnica dell'integrazione di algoritmi PQC in architetture moderne, evidenziando il ruolo cruciale della crypto-agility e dell'approccio ibrido come strategia di difesa proattiva per la sicurezza delle infrastrutture IT nel prossimo decennio.

Ringraziamenti

“A quella voce nella mia testa che non mi consentirà mai di fermarmi”.

Durante tutto il percorso triennale sono cresciuto tanto e ho imparato alcuni tra i valori personali più importanti per vivere al meglio la mia vita. Ho imparato che la maggior parte delle volte non serve correre se non si è pronti a farlo, talvolta è meglio semplicemente camminare preparandosi per quando ci sarà poi la reale necessità di correre. Ho a cuore ricordare che la vita, infatti, è molto più che una banale corsa al traguardo e quando una persona osserva solo il fine ultimo, il suo obiettivo, perde di vista quello che è il percorso lungo e tortuoso che c'è da fare per raggiungerlo, molto spesso è proprio lì infatti che si cela il senso vero e proprio del perché si sta facendo quel qualcosa in quel determinato istante. Mentirei spudoratamente se dicessi che questo percorso è stato semplice o che ho trovato la strada già spianata e pronta da percorrere, ma di una cosa invece sono certo, questo traguardo mi ha fatto capire cosa significa vivere, ed è molto più che un posto di lavoro, uno stipendio, una casa, una macchina o via dicendo. Si vive anche camminando, ed io, un passo alla volta raggiungerò la luna. Ringrazio quindi per primo me stesso per essere stato la prima persona a credere che **tutto ciò che si vuole fare si può fare**. In questo percorso non mi sono mai sentito davvero solo, accanto a me ho trovato sempre persone amiche e familiari pronti a permettermi di apprendere, **soprattutto dai miei errori**, ed è per questo che io porterò sempre dentro di me una piccola parte di ognuno di loro. Una parola in più va certamente spesa per **mia madre** e **mia sorella** che hanno sempre aiutato in maniera silente a ristabilire il mio umore anche nei momenti più bui e che con pazienza hanno aiutato sempre con ciò che io, spesso per motivi didattici, non sono stato presente per fare quotidianamente. **A tutti i miei colleghi** rivolgo i miei più sentiti ringraziamenti per i bei momenti vissuti insieme nel contesto universitario, le nostre risate sono state sicuramente la fonte più sincera di gioia che ho avuto il piacere di percepire in questo ateneo, per non parlare poi della condivisione dei nostri successi agli esami, tutti momenti che non dimenticherò mai e che ripercorrerei ben volentieri, auguro a tutti noi un futuro prospero. Ringrazio, infine, **i miei zii e i miei nonni**, figure di riferimento costanti che mi hanno guidato con saggezza, permettendomi di maturare scelte consapevoli

e offrendomi sempre un prezioso confronto. Ciò detto sono definitivamente pronto in quanto individuo ad affrontare tutte le ardue ma altrettanto entusiasmanti sfide per cui il mio ruolo da risolutore di problemi è indicato. Ma più di tutto sono pronto ad aiutare a fare del bene e a produrre qualcosa di utile per questo universo. Grazie a tutti per l'attenzione, garantisco che non finisce qui.

Indice

1	Introduzione	6
1.1	Motivazioni ed obiettivi della tesi	6
1.2	Contributi ed Innovazione della Tesi	7
1.3	Struttura della Tesi	8
2	Il Quantum Computing: Principi e Implicazioni per la Sicurezza	9
2.1	Cenni storici e nascita del quantum computer	9
2.2	Fondamenti di computazione quantistica	9
2.2.1	Il Qubit e le sue proprietà: sovrapposizione ed entanglement	10
2.2.2	Rappresentazione matematica del qubit	11
2.2.3	Il modello circuitale e le porte quantistiche	12
2.3	Perché il quantum computing è una minaccia per la crittografia?	14
2.3.1	L'algoritmo di Shor e la fattorizzazione	14
2.3.2	L'algoritmo di Grover e la ricerca in database non strutturati	15
2.3.3	Lo stato dell'arte: limiti attuali e prospettive future	15
3	Crittografia Classica vs Crittografia Post-Quantum (PQC)	17
3.1	Algoritmi crittografici classici a rischio	17
3.1.1	Algoritmi di firma digitale	17
3.1.2	Algoritmi di stabilimento e incapsulamento delle chiavi (KEM)	18
3.1.3	Algoritmi di crittografia simmetrica e il loro livello di resilienza	18
3.2	Componenti e tecnologie crittografiche da migrare	19
3.2.1	Standard di protocolli di rete	19
3.2.2	Librerie crittografiche software	19
3.2.3	Hardware crittografico (HSM) e infrastrutture PKI	19
3.2.4	Applicazioni e servizi IT	20
3.3	Soluzioni Post-Quantum (PQC) proposte dal NIST	20
3.3.1	Nuovi standard per la firma digitale	21
3.3.2	Nuovi standard per il key encapsulation	24
3.3.3	Rafforzamento della crittografia simmetrica e delle funzioni hash	25

3.3.4	Migrazione verso i nuovi componenti PQC	25
3.3.5	Approcci ibridi come strategia di transizione	27
4	Problematiche e Sfide nella Migrazione verso il Post-Quantum	29
4.1	Lo stato dell'arte dell'hardware quantistico: l'era NISQ e il percorso verso la fault-tolerance	29
4.2	Complessità degli algoritmi PQC	31
4.2.1	Complessità matematica	32
4.2.2	Impatto computazionale su CPU, memoria e rete	32
4.3	Sfide architetturali e di integrazione	32
4.3.1	Impatto su sistemi legacy e dispositivi IoT (risorse limitate)	34
4.4	Nuovi vettori di attacco	34
4.4.1	L'attacco 'Store Now, Decrypt Later' (SNDL)	35
4.4.2	Vulnerabilità side-channel nelle implementazioni PQC	35
5	Attività Pratica: Implementazione di un Prototipo di Migrazione PQC	37
5.1	Contesto e piano di lavoro: il progetto AstroVerse	37
5.2	Scelte tecnologiche	38
5.3	Progettazione del servizio di crittografia ibrida (HybridCryptoService)	38
5.3.1	Generazione delle chiavi (ECC e ML-KEM)	38
5.3.2	Processo di cifratura ibrida (ECDH + ML-KEM + AES-GCM)	38
5.3.3	Processo di decifratura	39
5.4	Modifiche al back-end per supportare la PQC	39
5.4.1	Adeguamento del modello dati (Entity User) e della persistenza	39
5.4.2	Integrazione del servizio ibrido nel flusso di registrazione e di modifica utente	40
5.4.3	Implementazione di un componente per la migrazione automatica degli utenti legacy	40
5.5	Modifiche al front-end (Vue.js) per il feedback utente	40
5.6	Risultati e considerazioni sull'implementazione pratica	40
6	Conclusioni e Sviluppi Futuri	42
6.1	Considerazioni finali sullo stato della migrazione e l'urgenza di agire	42
6.2	Sensibilizzazione all'adozione di standard PQC e approcci ibridi	43
6.3	Difficoltà incontrate: il 'Quantum Hype' e le limitazioni dell'hardware attuale	43
6.4	Prospettive future: dall'analisi sperimentale offensiva al Quantum Machine Learning	44

Riferimenti Bibliografici	44
A Codice Sorgente	49
A.1 5.2	49
A.2 5.3	49
A.3 5.3.1	50
A.4 5.3.1	51
A.5 5.3.2	51
A.6 5.3.2	52
A.7 5.3.2	52
A.8 5.3.3	53
A.9 5.4	54
A.10 5.4	56
A.11 5.4.1	57
A.12 5.4.1	57
A.13 5.4.2	57
A.14 5.4.3	62
A.15 5.5	64

Capitolo 1

Introduzione

In questo capitolo verranno descritte le motivazioni e gli obiettivi della tesi, i contenuti in essa sviluppati e la relativa struttura.

1.1 Motivazioni ed obiettivi della tesi

Negli ultimi anni il forte impatto tecnologico subito con l'avvento del quantum computing e dei primi quantum computers ha presto permesso agli scienziati e ai ricercatori di dimostrarne **l'efficienza superiore (in termini di complessità temporale nella risoluzione di 'task') rispetto a un classico elaboratore e persino ad un calcolatore parallelo**. La notizia, diffusa rapidamente attraverso l'articolo della famosa testata giornalistica The Guardian: *'Google saluta la svolta del secolo: i quantum computer sorpassano l'abilità dei supercomputer'*[2], descrive come il quantum chip di Google, Willow, eseguendo un algoritmo di ricostruzione molecolare, da luogo a risultati che un supercomputer non sarebbe mai stato in grado di ottenere, questo perché un supercomputer può simulare molecole di piccole o medie dimensioni (in termini di struttura molecolare), ma non può risolvere esattamente la struttura elettronica di molecole complesse: il costo computazionale infatti, cresce in modo esponenziale per via del numero di stati quantistici di atomi o elettroni da considerare. Questo è solo un esempio delle varie dimostrazioni del potenziale espresso da questa nuova tecnologia, Google afferma che la tecnica quantum è stata circa 13000 volte più veloce rispetto a quella applicata sulla migliore configurazione classica nota per un supercomputer. Tuttavia Google stessa sottolinea che questo risultato, nonostante dimostri chiaramente ciò che è capace di fare il quantum computing, rappresenta soltanto un importante traguardo (milestone). Siamo dunque ancora ben lontani dalla stabilità dei quantum computer, il chip Willow infatti, come molti altri, non è ancora totalmente fault-tolerant (i qubit restano cioè soggetti a errori di misurazione, interferenze e fenomeni di decoerenza, [vedi 2.3.3]). Per una 'vera' applicazione commerciale del quantum computer servirebbero processori con decine di migliaia di qubit **stabili**. L'evoluzione di queste tecnologie tuttavia apre interrogativi cruciali: se un computer quantistico potesse essere utilizzato per decifrare messaggi o violare dati sensibili in tempi estremamente brevi, la sicurezza dei sistemi crittografici tradizionali sarebbe

compromessa. Sorge dunque spontanea la domanda: **abbiamo ancora abbastanza tempo per prepararci alle minacce derivanti dalla futura immissione sul mercato di potenti computer quantistici?** Il teorema di Mosca sottolinea l'urgenza di migrare verso algoritmi post-quantistici introducendo una linea temporale semplice ma efficace: se 'X' rappresenta il numero di anni per cui i dati devono rimanere sicuri, e 'Y' è il tempo stimato necessario per completare la transizione, allora le organizzazioni devono iniziare a migrare verso algoritmi post-quantistici prima che $X + Y$ superi il tempo previsto 'Z' per la costruzione di un computer quantistico crittograficamente rilevante (CRQC)[3].

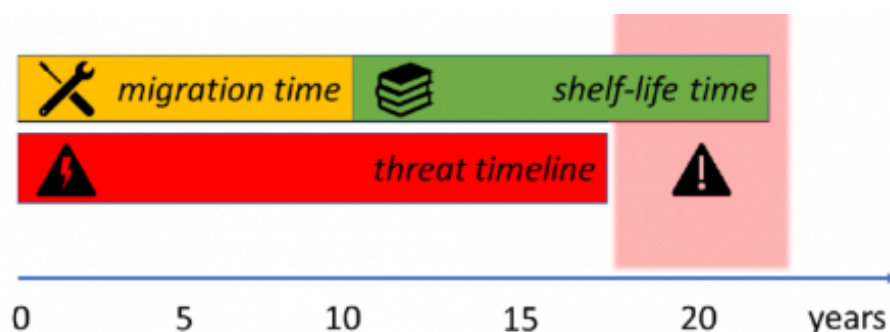


Figura 1.1: Rappresentazione grafica del teorema di Mosca: in giallo Y, in verde X, in rosso Z

Il NIST (National Institute of Standards and Technology), con il documento ufficiale NIST IR 8547[4] esprime il 2035 come data massima oltre la quale i sistemi che non implementano algoritmi di cifratura post-quantum (indicati come PQC) diventeranno vulnerabili ad attacchi. Questa data, riflette l'urgenza della transizione a metodi di crittografia che possano tollerare le future minacce quantum. L'obiettivo principale di questo lavoro di tesi è quindi **analizzare le sfide e le criticità da affrontare per garantire la sicurezza dei dati sensibili durante il processo di migrazione, in attesa che le tecnologie PQC vengano definitivamente consolidate**. Dal punto di vista pratico, la tesi si pone l'obiettivo di sviluppare un crittosistema di incapsulamento delle chiavi (KEM) basato su un approccio PQC ibrido. Questo permetterà di **approfondire, attraverso uno use case reale, le difficoltà tecniche incontrate nel passaggio da un sistema di crittografia classica a uno post-quantum**.

1.2 Contributi ed Innovazione della Tesi

Tra i contenuti principali di questa tesi possiamo osservare:

- Una breve introduzione al quantum computing senza troppi dettagli matematici e fisici complessi, alle varie componenti, alla struttura, al funzionamento di esse e allo stato dell'arte;

- Un'analisi organica delle problematiche di migrazione che spazia dai diversi algoritmi fino alle infrastrutture più utilizzate in linea con i documenti rilasciati dal NIST;
- Uno studio delle vulnerabilità emergenti dalla tecnologia quantistica come attacchi 'SNDL' e 'Side-Channel' specifici per la PQC;
- L'implementazione pratica e documentata di un approccio ibrido (ECC + ML-KEM) per la cifratura dei dati, dimostrando la fattibilità e le modifiche necessarie in un'applicazione full-stack (back-end Java/Spring Boot e front-end Vue.js).
- La progettazione e realizzazione di un componente software per la migrazione automatica di utenti legacy, affrontando il problema della compatibilità con i dati esistenti.

1.3 Struttura della Tesi

Questa tesi è strutturata come segue:

- Capitolo 2: Verranno introdotti i fondamenti del quantum computing spiegando il funzionamento dei qubit, la loro rappresentazione attraverso lo spazio degli stati, le principali porte logiche e le implicazioni sulla sicurezza informatica introducendo gli algoritmi di Shor e Grover e mostrando i limiti dei quantum computers esistenti;
- Capitolo 3: Verrà mostrato il confronto fra gli algoritmi di crittografia classica e post-quantum, analizzando gli algoritmi a rischio e le componenti infrastrutturali da aggiornare (protocolli di rete, PKI, librerie). Verranno poi presentate e analizzate le soluzioni di PQC proposte dal NIST;
- Capitolo 4: Verranno mostrate le sfide pratiche del processo di migrazione, analizzando lo stato attuale dei computer quantistici (era NISQ), la complessità degli algoritmi PQC e il loro impatto su sistemi legacy e IoT. Verranno inoltre approfonditi gli attacchi quantistici come lo SNDL e quelli side-channel;
- Capitolo 5: Verrà illustrata l'attività pratica svolta durante il tirocinio interno nel laboratorio ARClab ([Advanced Research On Cryptography Security](#)), mostrando nel dettaglio l'implementazione del servizio di PQC ibrida, le modifiche alla persistenza, ai servizi e al front-end di un'applicazione web, inclusa la componente per la migrazione degli utenti legacy;
- Capitolo 6: verranno discusse le conclusioni finali, le difficoltà incontrate, i futuri sviluppi e le prospettive future per la ricerca e l'adozione della crittografia post-quantistica.

Capitolo 2

Il Quantum Computing: Princìpi e Implicazioni per la Sicurezza

In questo capitolo verrà illustrato, senza entrare troppo nel merito matematico dell'argomento, quello che è il concetto di quantum. Attraverso una panoramica sui qubit andremo poi a spiegare che cos'è un quantum computer, come opera e quali sono le sue vulnerabilità.

2.1 Cenni storici e nascita del quantum computer

Tutto nacque nei primi anni '80, quando il fisico Paul Benioff propose il modello quantistico della macchina di Turing, dimostrando la possibilità del calcolo reversibile, ovvero mostrando come, lo stato iniziale del sistema può sempre essere ricostruito a partire dallo stato finale. In seguito il fisico Richard Feynman osservò che non tutti i fenomeni della fisica quantistica potevano essere simulati in modo efficiente su un computer classico, in particolare il fenomeno dell'entanglement quantistico (ovvero come due particelle, anche se separate spazialmente, mantengono uno stato comune). Feynman propose allora l'idea di un simulatore quantistico universale, cioè una macchina che sfruttasse direttamente le leggi quantistiche per simulare sistemi quantistici complessi. Queste idee posero il fondamento per il concetto moderno di computer quantistico.

2.2 Fondamenti di computazione quantistica

Un quantum computer è fondamentalmente un computer che utilizza le proprietà quantistiche della materia, come: la sovrapposizione degli stati (in questo caso dei bit 0 e 1) e l'entanglement quantistico; al fine di effettuare operazioni su dei dati. Andiamo ora a definire come avviene una computazione quantistica in un sistema. Una computazione quantistica si articola nelle seguenti fasi:

- Inizializzazione del registro a n qubit (nello stato $|0\dots 0\rangle$ o in uno stato generico $|\psi\rangle$);

- Trasformazione dello stato quantistico del registro
 - Mediante una sequenza di operazioni primitive (a 1 o 2 qubit) effettuate attraverso delle porte dette 'gates' (**vedi sezione 2.2.3**) che collettivamente eseguono la trasformazione del registro;
- Misurazione di alcuni o tutti i qubit del registro;
- Procedura di controllo
 - Applicata su tutto il programma per verificare quali passaggi eseguire;
 - Applicata sui risultati delle misurazioni per interpretarli.

A differenza di un calcolatore classico, il calcolatore quantistico opera con bit quantistici.

2.2.1 Il Qubit e le sue proprietà: sovrapposizione ed entanglement

Il qubit è l'unità di misura del quanto di informazione, cioè la porzione più piccola in cui una qualsiasi informazione codificata può essere scomposta. Un qubit gode di diverse proprietà derivanti dai postulati della meccanica quantistica:

Un qubit è rappresentato da un vettore di uno spazio di Hilbert $H = (H, \langle \cdot, \cdot \rangle)$. Così come il bit classico ammette due stati (0 e 1), altrettanto accade al qubit, tuttavia, grazie a ciò che emerge dal **primo postulato**, è anche possibile combinare linearmente i due stati (in notazione bracket) $|0\rangle$ e $|1\rangle$ per ottenere uno stato di sovrapposizione di entrambi contemporaneamente:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

dove a e b sono due numeri complessi in forma

$$x + iy$$

il quale quadrato del modulo rappresenta la probabilità di stato e tale per cui vale la proprietà di **normalizzazione**

$$|a|^2 + |b|^2 = 1$$

Grazie alle leggi della meccanica quantistica quindi, **il valore di un qubit può corrispondere a 0, a 1 oppure ad una combinazione di entrambi**. L'esito della misurazione dello stato di un qubit però, può essere soltanto $|0\rangle$ o $|1\rangle$. Per di più la misurazione del qubit stesso ne cambia lo stato riducendo la sovrapposizione in uno dei due valori. Quindi quando osserviamo un qubit per determinarne lo stato, esso si troverà sempre in una delle due configurazioni del bit classico. Questo risultato determina che **utilizzando un qubit è possibile ottenere la stessa quantità di informazione rappresentabile con un bit classico**. Possiamo quindi immaginare un bit

classico come il lancio di una moneta che una volta a terra, mostra il risultato di una delle due facce, invece possiamo immaginare il qubit come una moneta che una volta a terra continua a girare su se stessa senza arrestarsi, finché qualcuno non ne blocca la rotazione, mostrando il risultato. Il qubit si trova, quindi, finché non ne viene misurato lo stato, in uno stato appellato dalla meccanica quantistica come **'superposizione'**. Inoltre combinando più qubit tra loro (attraverso ciò che viene chiamato 'prodotto tensoriale') è possibile osservare il fenomeno di intreccio o **'entanglement'**, ovvero due qubit perdono la natura di stato quantistico individuale per formare un'unità di coppia. Al verificarsi di tale condizione gli stati individuali dei qubit si influenzano tra loro[5].¹

2.2.2 Rappresentazione matematica del qubit

Lo stato di un qubit può essere considerato come un punto (quindi una sovrapposizione complessa dei vettori ket $|0\rangle$ e $|1\rangle$) sulla sfera unitaria di Bloch tale che:

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle$$

con $0 \leq \theta \leq \pi$ e $0 \leq \phi < 2\pi$

¹Esistono anche sistemi a più stati rispetto al qubit, in generale un sistema a d-stati è chiamato qudit

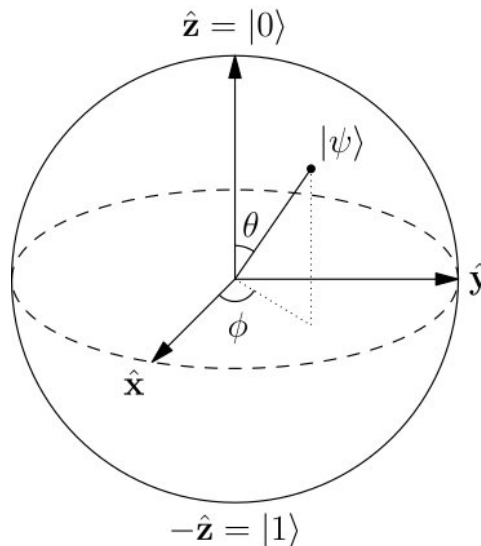


Figura 2.1: Sfera di Bloch che rappresenta lo spazio degli stati di un qubit

Dove θ rappresenta l'altezza sulla sfera (spin su o giù) e quantifica la distanza del qubit dallo stato 0 o dallo stato 1, mentre ϕ determina la fase relativa, ovvero la rotazione rispetto al piano x-y. I punti sulla superficie della sfera rappresentano gli stati 'puri' di un qubit. Sta ai programmatori controllare esplicitamente gli effetti quantistici:

- Sovrapposizione: questo qubit dovrebbe essere ruotato di questa quantità in questa direzione;
- Entanglement: questo qubit dovrebbe ruotare condizionatamente (in funzione di un altro qubit).

2.2.3 Il modello circuitale e le porte quantistiche

In analogia con l'architettura dei calcolatori classici, anche i computer quantistici si basano su un modello di computazione circuitale, in cui le operazioni elementari sui qubit vengono eseguite attraverso i cosiddetti 'gate' quantistici.

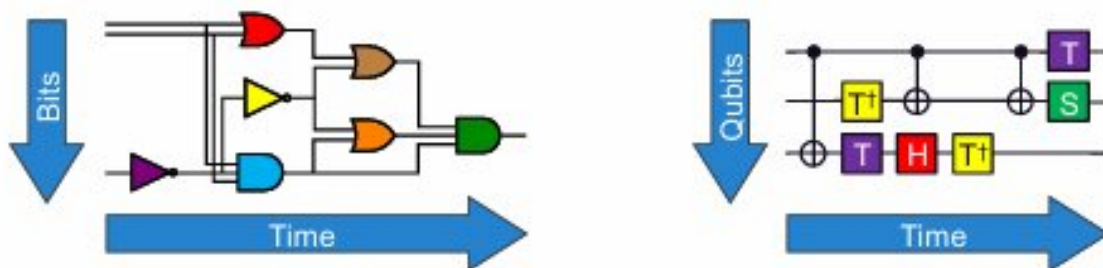


Figura 2.2: Differenze tra circuiti che operano su bit e su qubit in relazione al tempo

I circuiti quantistici devono essere reversibili (devono cioè fornire n input ed n output per ogni gate e viceversa), inoltre richiedono logica combinatoria. Nei computer quantistici, le porte quantistiche sono l'equivalente delle porte logiche classiche, ma agiscono su qubit invece che su bit. Ogni porta è un'operazione unitaria che trasforma lo stato dei qubit, preservandone la normalizzazione (**vedi sopra**). Le porte fondamentali consentono di creare sovrapposizione, entanglement e di manipolare le fasi relative, caratteristiche essenziali per l'elaborazione quantistica. Di seguito vengono riportate alcune delle più importanti:

- Porte X, Y e Z (Pauli): manipolano i qubit. La X funge da NOT ruotando di π radianti attorno all'asse x, la Y ruota di π radianti attorno all'asse y modificando la fase relativa, la Z ruota di π radianti attorno all'asse z modificando la fase relativa;
- Porta H (Hadamard): crea sovrapposizione tra 2 qubit, essenziale per il parallelismo;
- Porte T ed S: modificano solo la fase relativa;
- Porta SWAP: scambia i valori dei due qubit
- Porta CNOT: genera entanglement tra i 2 qubit, creando una configurazione $|\chi\psi\rangle$ dove χ viene detto 'bit di controllo' e ψ viene detto 'bit bersaglio'. Esempio se ho la configurazione $|10\rangle$ il mio bit di controllo fa eseguire un bit flip al bersaglio, la configurazione dopo la porta CNOT diverrà $|11\rangle$

Porta	Azione	Matrice
X (Pauli-X)	Bit-flip: $ 0\rangle \leftrightarrow 1\rangle$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Y (Pauli-Y)	Bit + phase-flip	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Z (Pauli-Z)	Phase-flip: $ 1\rangle \rightarrow - 1\rangle$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
H (Hadamard)	Sovrapposizione: $ 0\rangle \rightarrow \frac{ 0\rangle+ 1\rangle}{\sqrt{2}}$	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
T	Phase gate: $ 1\rangle \rightarrow e^{i\pi/4} 1\rangle$	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
S	Phase gate: $ 1\rangle \rightarrow i 1\rangle$	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
SWAP	Scambia i valori dei due qubit $ ab\rangle = ba\rangle$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
CNOT	Controlled-NOT a 2 qubit: inverte il qubit bersaglio se il bit di controllo è uguale a 1	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

Tabella 2.1: Principali porte quantistiche e loro azione sui qubit espressa sotto forma di matrice.

2.3 Perché il quantum computing è una minaccia per la crittografia?

Anche se attualmente non esistono computer quantistici con rilevanza crittografica (CRQCs, cryptographically relevant quantum computers) che minaccino i livelli di sicurezza dei sistemi già esistenti, sarà necessario un tempo significativo per passare ai nuovi algoritmi post-quantistici: 'Le migrazioni crittografiche del passato hanno richiesto oltre un decennio, e questa migrazione più complessa probabilmente richiederà almeno altrettanto tempo' - afferma il NIST[4]. Ciò significa che, anche se i computer quantistici sono, secondo analisti esperti (il CEO di Google[6] e le società di analisi Gartner, McKinsey & Co.[7][8][3]), probabilmente ad almeno un decennio di distanza, le organizzazioni devono iniziare oggi la migrazione alla crittografia post-quantistica per evitare che i loro dati cifrati vengano esposti una volta che i computer quantistici diventeranno operativi in futuro. Questa minaccia, definita 'store now, decrypt later' (**vedi sezione 4.4.1**), sottolinea la necessità di agire immediatamente, soprattutto per i dati con sensibilità a lungo termine, come segreti di stato o cartelle cliniche. Garantire la sicurezza oggi la salvaguarderà per il futuro. Storicamente, la sicurezza fornita da un algoritmo era definita in termini di quantità di lavoro (cioè, il numero di operazioni) necessario per violarlo (cioè, un algoritmo ha s bit di sicurezza se violarlo richiede 2^s operazioni di qualche tipo, dove $s = 112, 128, 192$ o 256). Tuttavia, esistono significative incertezze nella stima della sicurezza dei crittosistemi post-quantum, data la difficoltà di prevedere accuratamente le caratteristiche di prestazione dei futuri computer quantistici, come il loro costo, la velocità e la capacità di memoria.

2.3.1 L'algoritmo di Shor e la fattorizzazione

La fattorizzazione di numeri primi è uno dei problemi matematici più accreditati, nonché fondamento della sicurezza crittografica moderna, che sfrutta la difficoltà di risolvere un problema semplice da verificare ma difficile da risolvere. La fattorizzazione in crittografia è un processo fondamentale alla base di molti sistemi di sicurezza informatica e la sua importanza deriva dal fatto che scomporre un intero molto grande in fattori primi è computazionalmente molto difficile, mentre il prodotto di due numeri primi grandi è facile da calcolare[9]. Questa asimmetria è ciò che garantisce la sicurezza del sistema. In RSA, ad esempio, si generano due numeri primi grandi, p e q , e si calcola il loro prodotto $n = p \cdot q$. Il numero n viene reso pubblico (modulo), mentre p e q rimangono segreti. La chiave privata è derivata da n e da $\phi(n) = (p - 1)(q - 1)$, che richiede la conoscenza di p e q . Chiunque voglia violare il sistema deve fattorizzare n , un'operazione che, con numeri di dimensioni attuali (2048 bit), richiede tempi impraticabili con i computer attuali in quanto, non esiste un algoritmo efficiente (polinomiale) per la fattorizzazione di numeri interi grandi. Il metodo più generale è la divisione per tentativi, che richiede di provare tutti i numeri primi fino a $\sqrt{n}$. Anche algoritmi più avanzati come il Fermat factorization o il General Number Field Sieve (GNFS) diventano impraticabili per numeri con centinaia di

cifre. Con l'avvento di un CRQC però, fattorizzare numeri interi potrebbe diventare un problema risolvibile in tempo polinomiale attraverso l'utilizzo dell'algoritmo di Shor. Si tratta di un algoritmo in grado, se eseguito su un calcolatore quantistico sufficientemente potente e stabile, di calcolare il periodo di una funzione (l'intervallo più piccolo dopo il quale la funzione ripete esattamente gli stessi valori) attraverso l'algoritmo Quantum Fourier Transform (QFT) per poi fattorizzare i numeri primi attraverso le informazioni ottenute dal calcolo del periodo.

2.3.2 L'algoritmo di Grover e la ricerca in database non strutturati

Un altro pericolo derivante dall'avvento dei CRQC e del loro utilizzo per violare la sicurezza dei dati è noto come algoritmo di Grover. Questo tipo di algoritmo quantistico di ricerca può trovare un elemento specifico all'interno di un enorme database non strutturato (per esempio, in uno scenario di attacco, nello spazio computazionale di possibili chiavi) in modo molto più veloce rispetto a un computer classico. L'algoritmo è così veloce perché sfrutta le proprietà quantistiche: l'algoritmo non 'legge' fisicamente i dati come un database, ma opera su una sovrapposizione di stati che rappresentano gli indici del database. Esso consiste di 3 fasi principali:

1. Superposizione: l'algoritmo considera tutte le possibili decifature e combinazioni utilizzando la superposizione dei qubit;
2. Marcatura (o oracolo): una funzione speciale (utilizzata molto spesso in ambito di algoritmi di ricerca post-quantum) detta 'oracolo' marca la risposta corretta, in termini di qubit utilizza un gate per invertirne la fase (**vedi Pauli gate Z**);
3. Amplificazione: un altro gate amplifica l'intensità della risposta marcata e diminuisce tutte le altre.

In questo modo iterando per un numero calcolato i passi 2 e 3 la probabilità di trovare una risposta diventa molto alta, sviluppando una proprietà detta **quadratic speedup**.

2.3.3 Lo stato dell'arte: limiti attuali e prospettive future

Come già precedentemente accennato, i quantum computer attuali e quelli ancora in via di sviluppo, non sono assolutamente perfetti e anzi, soffrono alcune limitazioni fondamentali che impediscono, allo stato attuale, il raggiungimento di una piena tolleranza ai guasti (fault tolerance). Tali limitazioni definiscono l'attuale fase di sviluppo della tecnologia quantistica, nota in letteratura come **era NISQ** (*Noisy Intermediate-Scale Quantum*). Il termine, introdotto da John Preskill nel 2018 che viene ritrattato in particolare nel documento[10] (che verrà discusso approfonditamente nella **sezione 4.1**), fa riferimento a dispositivi di 'scala intermedia' (composti da un numero di qubit compreso tra 50 e qualche centinaio) che, pur possedendo una potenza computazionale teoricamente molto superiore ai supercomputer classici per specifici task, sono definiti 'rumorosi' (*noisy*). Questa caratteristica implica che i processori attuali non

dispongono ancora di meccanismi robusti di correzione degli errori quantistici (*Quantum Error Correction, QEC*): di conseguenza, la fedeltà delle operazioni decade rapidamente all'aumentare della profondità del circuito, limitando l'applicabilità dei dispositivi odierni ad algoritmi di breve durata e tolleranti alle imprecisioni. Vediamo ora i principali tipi di problematiche che si possono verificare sui qubit.

Decoerenza Quantistica

Il principale ostacolo alla realizzazione di calcoli quantistici complessi è rappresentato dalla **decoerenza**. I qubit, a differenza dei bit classici, devono mantenere il loro stato di sovrapposizione coerente per tutta la durata dell'algoritmo. Tuttavia, qualsiasi interazione non controllata con l'ambiente esterno (vibrazioni termiche, radiazioni elettromagnetiche) porta a una perdita di informazione quantistica. In termini tecnici, la decoerenza viene caratterizzata da due tempi di rilassamento fondamentali:

- **Tempo di rilassamento energetico (T_1):** Il tempo necessario affinché un qubit nel livello eccitato $|1\rangle$ decada spontaneamente allo stato fondamentale $|0\rangle$.
- **Tempo di dephasing (T_2):** Il tempo durante il quale la fase relativa della sovrapposizione quantistica rimane intatta prima di essere modificata dalle fluttuazioni ambientali.

Quando la durata di un algoritmo supera questi tempi caratteristici, il risultato del calcolo diviene puramente casuale e privo di significato.

Crosstalk e Rumore Ambientale

Sebbene l'interferenza sia ampiamente impiegata nel funzionamento degli algoritmi quantistici, essa diviene una problematica quando si manifesta sotto forma di *interferenza indesiderata* o **Crosstalk**. Questo fenomeno si verifica quando l'operazione logica eseguita su un determinato qubit influenza accidentalmente lo stato dei qubit adiacenti, a causa della difficoltà di isolare perfettamente le interazioni elettromagnetiche su scala microscopica. Tale 'rumore correlato' è particolarmente insidioso poiché, a differenza degli errori casuali singoli, può propagarsi a catena attraverso il circuito, rendendo inefficaci le tecniche di mitigazione degli errori standard.

Errori di Misurazione (Readout Errors)

L'ultima fase di qualsiasi algoritmo quantistico prevede la misurazione dei qubit per estrarre un risultato classico (0 o 1). Questa operazione non è esente da errori, noti in letteratura come *SPAM errors* (State Preparation And Measurement). Attualmente, la fedeltà di lettura (*readout fidelity*) è spesso inferiore rispetto alla fedeltà delle porte logiche: ciò significa che, anche se il calcolo quantistico è stato eseguito correttamente, il dispositivo di misurazione potrebbe interpretare erroneamente lo stato finale (ad esempio, leggendo un $|0\rangle$ come un $|1\rangle$), falsando l'output complessivo del sistema.

Capitolo 3

Crittografia Classica vs Crittografia Post-Quantum (PQC)

In questo capitolo andremo a descrivere le principali famiglie di algoritmi classici a rischio e a confrontarli con le nuove proposte post-quantum.

3.1 Algoritmi crittografici classici a rischio

I nuovi standard di elaborazione federali prodotti dal NIST specificano un ampio insieme di primitive, algoritmi e schemi crittografici, inclusi molti crittosistemi a chiave pubblica che saranno deprecati e non più ammessi all'utilizzo dopo la transizione.

3.1.1 Algoritmi di firma digitale

Gli algoritmi di firma digitale sono utilizzati per fornire autenticazione dell'identità (il legame tra la persona che utilizza la firma e la firma stessa), dell'integrità (il documento dopo essere stato firmato non deve essere modificato, pena l'invalidità della firma), della fonte (attraverso la conferma dell'Identity Provider¹) e supporto per la non ripudiabilità (l'IDP non può negare di aver applicato la firma digitale al documento). Gli algoritmi di firma digitale vengono utilizzati insieme a funzioni hash o a funzioni a uscita estendibile (XOF)² per firmare messaggi di lunghezza arbitraria. Attualmente per la firma digitale viene utilizzato un algoritmo con schema di stabilimento delle chiavi basato su curve ellittiche (ECDSA). Tuttavia questo tipo di algoritmo è vulnerabile all'**algoritmo di Shor**.

¹L'ente che fornisce il supporto all'utilizzo dell'identità digitale, denotato anche come IDP

²Le funzioni hash sono noti algoritmi matematici che mascherano e riducono un input ad una lunghezza arbitraria creando ciò che viene detto 'hash' o 'digest', le XOF sono simili fatta eccezione per l'output che non avrà una lunghezza fissa

3.1.2 Algoritmi di stabilimento e incapsulamento delle chiavi (KEM)

Lo stabilimento delle chiavi è il mezzo attraverso il quale due chiavi crittografiche vengono generate e fornite agli enti autorizzati a utilizzarle. Gli schemi di stabilimento delle chiavi attualmente in uso sono:

- Schemi di stabilimento delle chiavi Pair-Wise³ utilizzando la crittografia basata sul calcolo del logaritmo discreto (cifrario Diffie-Hellman e algoritmo ECDH che lo implementa);
- Schemi di stabilimento delle chiavi Pair-Wise utilizzando la crittografia basata su fattorizzazione degli interi (l'algoritmo RSA è il più utilizzato in questo ambito).

Entrambi questi schemi sono facilmente decifrabili utilizzando l'**algoritmo di Shor**.

3.1.3 Algoritmi di crittografia simmetrica e il loro livello di resilienza

Gli algoritmi a chiave simmetrica (a volte chiamati algoritmi a chiave segreta) utilizzano una singola chiave sia per applicare la protezione crittografica, sia per rimuovere o verificare la protezione (cioè, la stessa chiave viene utilizzata per un'operazione crittografica e per la sua inversa). Ad esempio, la chiave utilizzata per cifrare i dati (cioè per applicare la protezione) viene anche utilizzata per decifrare i dati cifrati (cioè per rimuovere la protezione). Nel caso della cifratura, i dati originali sono chiamati testo in chiaro, mentre la forma cifrata dei dati è chiamata testo cifrato. La chiave deve essere mantenuta segreta affinché i dati rimangano protetti. Sono state approvate con il tempo diverse classi di algoritmi a chiave simmetrica:

- Basate su algoritmi di cifratura a blocchi (es. AES)
- Basate sull'uso di funzioni hash (es. codici di autenticazione del messaggio con chiave generata mediante funzione hash)

Gli algoritmi di crittografia simmetrica hanno molteplici applicazioni:

- Cifrari a blocchi
- Funzioni di hash
- Crittografia che utilizza modalità operative a blocchi
- Autenticazione dei dati utilizzando modalità operative a blocchi
- Autenticazione dei dati utilizzando costruzioni con chiave hash
- Derivazione delle chiavi
- Incapsulamento delle chiavi

³Garantire una chiave univoca e segreta per ogni coppia

- Generazione di bit casuali

Questo tipo di algoritmi sono meno vulnerabili e non prevedono la necessità di passare a standard diversi di PQC (**vedi 3.3.3 per ulteriori approfondimenti**).

3.2 Componenti e tecnologie crittografiche da migrare

Molte applicazioni includono componenti basate su protocolli e tecnologie di sicurezza standardizzate che dovranno migrare per resistere agli attacchi quantum. Inoltre, le applicazioni sono costruite su librerie crittografiche software che forniscono le implementazioni degli algoritmi crittografici o un'interfaccia ai moduli crittografici hardware. Tutte le librerie crittografiche software e i moduli crittografici hardware utilizzati da un'applicazione dovranno essere revisionati per supportare algoritmi post-quantum. Le applicazioni possono anche dipendere da componenti infrastrutturali, che dovranno essere aggiornate per supportare gli algoritmi prima che le applicazioni possano migrare all'uso degli stessi.

3.2.1 Standard di protocolli di rete

I protocolli di rete insieme agli standard per la sicurezza definiscono le regole per lo scambio di dati sulle reti e garantiscono una comunicazione sicura e affidabile. Esempi includono Transport Layer Security (TLS), Secure Shell (SSH), Internet Protocol Security (IPsec) e Cryptographic Message Syntax (CMS). Questi protocolli e tecnologie di sicurezza si basano spesso su algoritmi crittografici classici che sono vulnerabili agli attacchi quantistici. Aggiornarli per incorporare algoritmi PQC è essenziale per mantenere la riservatezza e l'integrità dei dati.

3.2.2 Librerie crittografiche software

Le librerie crittografiche software sono raccolte di algoritmi e protocolli crittografici implementati in software per fornire funzioni crittografiche essenziali alle applicazioni. OpenSSL, Bcrypt, Libsodium e la Java Cryptography Architecture (JCA) sono alcuni esempi di librerie crittografiche utilizzate per fornire supporto crittografico alle applicazioni. Queste librerie devono integrare algoritmi PQC poiché solo in tal modo si garantisce che gli sviluppatori possano avere accesso a funzioni crittografiche resistenti ai computer quantistici senza dover implementare personalmente algoritmi complessi.

3.2.3 Hardware crittografico (HSM) e infrastrutture PKI

I moduli crittografici hardware forniscono ambienti sicuri per eseguire operazioni crittografiche e per memorizzare chiavi sensibili. Sono utilizzati in varie applicazioni, dalla sicurezza delle infrastrutture server alla protezione delle chiavi crittografiche sui dispositivi personali. I moduli hardware devono pertanto essere aggiornati o riprogettati per supportare gli algoritmi PQC, che

spesso richiedono chiavi di dimensioni maggiori o requisiti computazionali diversi. Le Public Key Infrastructure (PKI) sono sistemi che si occupano di gestire certificati digitali e coppie di chiavi pubblica-privata per consentire comunicazioni sicure e autenticazione attraverso le reti. Le altre componenti dell'infrastruttura includono le autorità di certificazione (CA), le autorità di registrazione, i sistemi di gestione delle chiavi e i servizi di directory. I componenti PKI devono essere aggiornati per emettere, distribuire e gestire certificati che utilizzano algoritmi PQC e per firmare certificati e informazioni sullo stato di revoca utilizzando algoritmi PQC.

3.2.4 Applicazioni e servizi IT

Le applicazioni e i servizi IT comprendono un'ampia gamma di software e piattaforme utilizzati dalle organizzazioni, tra cui applicazioni web, database, strumenti di comunicazione, servizi cloud e software aziendali. Queste applicazioni si affidano alla crittografia per proteggere i dati, autenticare gli utenti e garantire transazioni sicure. Le applicazioni e i servizi devono essere modificati per supportare gli algoritmi PQC per la crittografia, le firme digitali e lo scambio di chiavi.

3.3 Soluzioni Post-Quantum (PQC) proposte dal NIST

Vediamo dunque adesso alcune delle risoluzioni proposte alle vulnerabilità elencate nelle sezioni 3.1 e 3.2. E' importante notare che alcune di queste soluzioni potrebbero inizialmente implementare algoritmi ibridi che combinano cioè sia algoritmi PQC sia standard odierni, almeno finché non venga completata del tutto la transizione, tuttavia le alternative ibride spesso complicano le implementazioni e le architetture sulle quali vengono utilizzate. Esse vengono pertanto considerate come alternative di passaggio verso un utilizzo esclusivo di algoritmi PQC. Saranno pertanto indicati come:

- **Accettabili:** gli algoritmi e la lunghezza e robustezza delle chiavi che sono approvate per l'uso in quanto resistenti a possibili attacchi quantum;
- **Deprecati:** gli algoritmi e le lunghezze delle chiavi o la loro robustezza che possono essere comunque utilizzati ma con dei rischi di sicurezza. Dunque l'utilizzatore di tali strumenti deve valutare questo potenziale rischio prima di procedere alla protezione dei dati;
- **Non consentiti:** gli algoritmi, le chiavi, i parametri e gli schemi che non sono più ritenuti consoni agli scopi protettivi;
- **Legacy:** gli algoritmi, le chiavi, i parametri e gli schemi protettivi che verranno utilizzati al solo scopo di processare informazioni già protette (ad esempio: per decifrare dati cifrati, o per verificare una firma digitale).

3.3.1 Nuovi standard per la firma digitale

Il problema principale degli algoritmi di firma digitale è quindi l'esecuzione dell'algoritmo di Shor su un computer quantistico crittograficamente rilevante⁴. Tra le proposte PQC per gli algoritmi di firma digitale troviamo:

Algoritmo di Firma Digitale Basato su Moduli e Reticoli (ML-DSA)

Il Module-Lattice-Based Digital Signature è un algoritmo che basa la sua sicurezza sulla difficoltà di risolvere problemi matematici su strutture chiamate reticoli cristallini (lattices). I reticoli sono griglie di punti multi-dimensionali la cui complessità geometrica crea problemi matematici. Si ritiene che questi problemi siano estremamente difficili da risolvere, sia per i computer classici che per quelli quantistici (incluso l'algoritmo di Shor) in quanto per trovare il punto più vicino in un reticolo cristallino bisogna conoscere una 'mappa' (che in questo caso costituisce la chiave segreta dell'algoritmo).

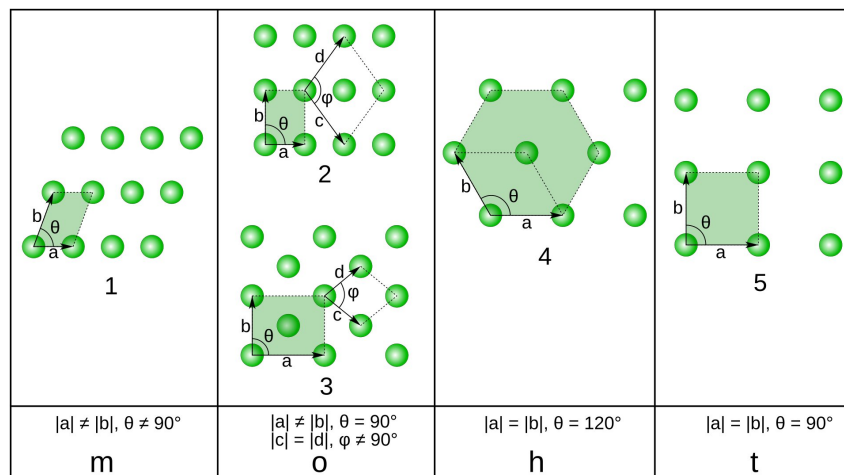


Figura 3.1: Esempi di reticoli (di Bravais) a 2 dimensioni

⁴Cioè un computer i cui qubit siano abbastanza stabili e potenti allo stesso tempo

Schema di Firma Digitale a Funzione Hash con e senza stato

Gli schemi di firma digitale basati su hash devono la loro sicurezza unicamente alla solidità di una funzione hash crittografica. La distinzione tra *stateful* (con stato) e *stateless* (senza stato) sta principalmente nel come viene gestita la One Time Signature (OTS), una chiave monouso che, come lascia ad intendere il nome, può essere utilizzata una sola volta in modo sicuro

- Nello schema con stato: il sistema memorizza le OTS utilizzate ed evita di riutilizzarle, dunque inizializza un sistema con un gran numero di chiavi (ad esempio un milione) e le aggrega in una struttura ad albero (detta Merkle Tree) per creare un'unica chiave pubblica 'master'. Per firmare il prossimo documento si utilizza la chiave monouso direttamente successiva e si aggiorna un contatore interno. Questo algoritmo nonostante l'utilizzo di una chiave piccola e quindi anche di un processo di firma molto veloce, è estremamente fragile, infatti la gestione dello stato rappresenta un punto debole in quanto se si perde traccia del contatore (in situazioni di ripristino o di criticità **vedi 4.3**) si rischia di utilizzare più volte la stessa OTS, portando ad un fallimento catastrofico della sicurezza e consentendo ad un possibile attaccante di riprodurre una firma falsificata. Il sistema di firma Leighton-Micali (LMS), l'eXtended Merkle Signature Scheme (XMSS) e l'HBS sono gli standard selezionati dal NIST e vengono utilizzati soprattutto in ambito di moduli di sicurezza hardware;
- Nello schema senza stato invece: Il sistema non ha necessità di memorizzazione poiché ogni operazione di firma è indipendente dalle altre. La struttura interna allo schema infatti non utilizza un contatore, bensì una chiave monouso scelta in modo pseudocasuale ma deterministico a partire dal messaggio e da un componente segreto (una componente della chiave privata del firmatario). Questo algoritmo quindi è ben più robusto, lo svantaggio tuttavia è dato dall'utilizzo di una firma generata significativamente più grande e lenta. Il sistema SPHINCS+ è stato selezionato come standard dal NIST e viene utilizzato maggiormente in ambiti dove la robustezza è più importante delle prestazioni.

Nella tipologia di algoritmi con schema a stato, poiché la chiave privata consiste in un insieme di OTS e data la necessità di memorizzare uno stato, la sicurezza dipende dal firmatario. Si sta valutando quindi un'alternativa che verrà rilasciata prossimamente dal NIST in un futuro documento FIPS⁵.

⁵Federal Information Processing Standards: documento che descrive standard di elaborazione dell'informazione federali (crittografia, interoperabilità, ecc.)

Famiglia di Algoritmi	Parametri	Transizione
ECDSA [FIPS186]	112 bit di forza di sicurezza	Deprecato dopo il 2030 Non consentito dopo il 2035
	≥ 128 bit di forza di sicurezza	Non consentito dopo il 2035
EdDSA [FIPS186]	≥ 128 bit di forza di sicurezza	Non consentito dopo il 2035
RSA [FIPS186]	112 bit di forza di sicurezza	Deprecato dopo il 2030 Non consentito dopo il 2035
	≥ 128 bit di forza di sicurezza	Non consentito dopo il 2035

Tabella 3.1: Algoritmi di firma digitale vulnerabili alla computazione quantistica

Famiglia di Algoritmi	Set di Parametri	Forza di Sicurezza
ML-DSA [FIPS204]	ML-DSA-44	128 bit
	ML-DSA-65	192 bit
	ML-DSA-87	256 bit
SLH-DSA [FIPS205]	SLH-DSA-SHA2-128[s/f] SLH-DSA-SHAKE-128[s/f]	128 bit
	SLH-DSA-SHA2-192[s/f] SLH-DSA-SHAKE-192[s/f]	192 bit
	SLH-DSA-SHA2-256[s/f] SLH-DSA-SHAKE-256[s/f]	256 bit
LMS, HSS [SP800208]	Con SHA-256/192 Con SHAKE256/192	192 bit
	Con SHA-256 Con SHAKE256	256 bit
XMSS, XMSS ^{MT} [SP800208]	Con SHA-256/192 Con SHAKE256/192	192 bit
	Con SHA-256 Con SHAKE256	256 bit

Tabella 3.2: Standard di algoritmi di firma digitale post-quantum proposti dal NIST

3.3.2 Nuovi standard per il key encapsulation

Data la facile decifrabilità dei meccanismi attuali attraverso l'uso dell'algoritmo di Shor in un possibile attacco 'store now, decrypt later', è stata effettuata una proposta di schema PQC da parte del NIST.

KEM basato su Modulo-Reticolato (ML-KEM)

Il Module Lattice-based Key Encapsulation Mechanism è un meccanismo di incapsulamento delle chiavi post-quantistico basato sui reticoli cristallini (**vedi Reticoli**). Il suo obiettivo è quello di permettere a due parti di stabilire una chiave segreta condivisa in modo sicuro su un canale insicuro. Il processo inizia con l'entità che desidera ricevere un segreto, che per semplicità indicheremo con X. X genera la sua chiave privata, che sarà costituita da un insieme di operazioni segrete. Per creare la chiave pubblica, X applica queste operazioni segrete alla struttura del reticolo (nota a tutti), ottenendo un punto esatto sulla griglia che viene poi spostato leggermente aggiungendo del 'rumore'⁶. Questo nuovo punto alterato costituisce la sua chiave pubblica. Per un estraneo sarà dunque impossibile risalire, dal solo punto alterato, alle operazioni segrete effettuate da X e quindi alla sua chiave privata. A questo punto, un'altra entità, Y, volendo stabilire un segreto con X, utilizza la sua chiave pubblica. Y genera la chiave segreta condivisa e la nasconde incapsulandola, ovvero aggiungendola come un ulteriore strato di rumore alla chiave pubblica di X. Il risultato di questa operazione è il testo cifrato (ciphertext), che viene spedito ad X. Infine, nella fase di deincapsulamento, X utilizzerà la sua chiave privata per annullare sia il suo rumore iniziale sia la struttura del reticolo dal ciphertext ricevuto. Ciò che rimane è solo il rumore aggiunto da Y, ovvero la chiave segreta condivisa, che X può recuperare facilmente eliminando il piccolo disturbo residuo.

Schema di Stabilimento Chiave	Parametri	Transizione
DH e MQV su Campo Finito [SP80056A]	112 bit di forza di sicurezza	Deprecato dopo il 2030 Non consentito dopo il 2035
	≥ 128 bit di forza di sicurezza	Non consentito dopo il 2035
DH e MQC su Curva Ellittica [SP80056A]	112 bit di forza di sicurezza	Deprecato dopo il 2030 Non consentito dopo il 2035
	≥ 128 bit di forza di sicurezza	Non consentito dopo il 2035
RSA [SP80056B]	112 bit di forza di sicurezza	Deprecato dopo il 2030 Non consentito dopo il 2035
	≥ 128 bit di forza di sicurezza	Non consentito dopo il 2035

Tabella 3.3: Schemi di scambio di chiavi vulnerabili alla computazione quantistica

⁶In termini matematici un disturbo, generato sommando un piccolo vettore di errore

Schema di Stabilimento Chiave	Set di Parametri	Forza di Sicurezza	Categoria di Sicurezza
ML-KEM [FIPS203]	ML-KEM-512	128 bits	1
	ML-KEM-768	192 bits	3
	ML-KEM-1024	256 bits	5

Tabella 3.4: Schemi di stabilimento di chiavi post-quantum proposti dal NIST

3.3.3 Rafforzamento della crittografia simmetrica e delle funzioni hash

Gli standard di crittografia simmetrica discussi dal NIST come: XOF (vedi piè di pagina 3.1.1), funzioni hash, cifrari a blocchi e altri standard crittografici elencati nella sezione 3.1.3, sono più resistenti ad attacchi quantum poiché tutti i primitivi simmetrici garantiscono almeno 128 bit di sicurezza. Si dispone pertanto, in misura preventiva, di abbandonare tutti i tipi di standard che utilizzano una quantità inferiore a 128 bit di sicurezza entro il 2030. Favorendo così la migrazione verso crittosistemi simmetrici più sicuri.

Cifrario a Blocchi	Varianti	Forza di sicurezza	Categoria di sicurezza
AES [FIPS197]	AES-128	128 bits	1
	AES-192	192 bits	3
	AES-256	256 bits	5

Tabella 3.5: Standard per cifrario a blocchi in uso e relativa sicurezza

Famiglia Algoritmo Hash/XOF	Varianti	Forza di Sicurezza (Collisione)	Categoria di Sicurezza (Collisione)	Forza di Sicurezza (Preimmagine)	Categoria di Sicurezza (Preimmagine)
SHA-1 [FIPS180]	SHA-1	80 bits	< 1	160 bits	1
SHA-2 [FIPS180]	SHA-224 SHA-512/224	112 bits	< 1	224 bits	3
	SHA-256 SHA-512/256	128 bits	2	256 bits	5
	SHA-384	192 bits	4	384 bits	5
	SHA-512	256 bits	5	512 bits	5
	SHA3-224	112 bits	< 1	224 bits	3
SHA-3 [FIPS202]	SHA3-256	128 bits	2	256 bits	5
	SHAKE128	128 bits	2	128 bits	2
	SHA3-384	192 bits	4	384 bits	5
	SHA3-512	256 bits	5	512 bits	5
	SHAKE256	256 bits	5	512 bits	5

Tabella 3.6: Funzioni Hash e XOFs con livelli di sicurezza pre e post migrazione (collisione e preimmagine)

3.3.4 Migrazione verso i nuovi componenti PQC

Per aggiornare questo tipo di standard bisognerà revisionare le specifiche dei protocolli affinché supportino i meccanismi PQC e nuovi metodi di autenticazione quantum-resistant. In alcuni

casi sarà sufficiente assegnare un identificatore al nuovo algoritmo; in altri, invece, saranno necessarie modifiche più significative per adattarsi alla maggiore complessità degli algoritmi PQC.

Un esempio interessante: ZK-STARK

Sono già in atto diversi studi per implementare nuove tecnologie autenticative post-quantum. Tra queste vi è ZK-STARK (Zero-Knowledge Scalable Transparent Argument of Knowledge), un tipo di protocollo crittografico di prova a conoscenza zero (zero-knowledge proof) progettato per essere altamente efficiente e sicuro anche contro la minaccia dei computer quantistici. A differenza di altre tecnologie simili come gli zk-SNARKs, gli ZK-STARKs sono stati sviluppati per superare due loro limitazioni fondamentali: la necessità di una 'trusted setup' (cerimonia di setup fidata) e la vulnerabilità agli attacchi quantistici. Basandosi su quanto detto sulla tesi: 'Post-Quantum Anonymous Credentials and ZK-STARKs'[11], le caratteristiche principali di queste tecnologie sono:

- **Sicurezza Post-Quantistica:** la loro sicurezza non si basa su problemi matematici vulnerabili ai computer quantistici (come la fattorizzazione o i logaritmi discreti su curve ellittiche). Invece, si fondano su primitive crittografiche più resistenti, come le funzioni di hash resistenti alle collisioni. Questo li rende una soluzione a lungo termine per la privacy e l'autenticazione;
- **Trasparenza (attraverso Transparent Setup):** gli ZK-STARKs non richiedono una 'trusted setup'⁷. Bensì essi utilizzano un processo di setup basato su casualità pubblicamente verificabile⁸, eliminando ogni necessità di fiducia e rendendo il sistema più sicuro e trasparente;
- **Scalabilità:** il tempo richiesto al 'prover' (colui che genera la prova) cresce in modo quasi-lineare rispetto alla complessità del calcolo da dimostrare, mentre il tempo per il 'verifier' (colui che verifica la prova) è polilogaritmico. Ciò significa che anche per calcoli molto complessi, la prova rimane relativamente piccola e veloce da verificare.

Per nozioni ulteriori su funzionamento, modello matematico e correttezza di questo algoritmo e dei suoi 'isotopi', si rimanda alla fonte principale indicata **nei riferimenti bibliografici**.

Librerie crittografiche per i software

Le librerie crittografiche per i software dovrebbero implementare gli algoritmi PQC standardizzati proposti nelle precedenti sezioni. Questa transizione comporta l'aggiunta di nuovi

⁷Fase necessaria per altri sistemi di autenticazione attraverso prova (es: ZK-SNARKs) in cui vengono generati parametri segreti. Se questi segreti vengono compromessi il sistema è a rischio

⁸Ovvero la cui verifica è attuabile anche dall'utente senza bisogno di IDP

algoritmi, l'ottimizzazione delle loro implementazioni per le prestazioni e l'assicurarsi che tali implementazioni siano sicure contro gli attacchi di tipo side-channel (**vedi 4.4.2**).

Hardware crittografico

Per migliorarne la sicurezza bisogna aggiornare il firmware dei dispositivi per gestire i nuovi algoritmi e garantire che i moduli hardware possano eseguire operazioni di calcolo quantistico e di crittografia resistente in modo efficiente mantenendo al contempo gli elevati standard di sicurezza previsti per questi dispositivi.

PKI e altre componenti infrastrutturali

Per migrare verso soluzioni resistenti è richiesto il supporto a nuovi algoritmi crittografici nei processi di emissione dei certificati e la modifica dei meccanismi di convalida e revoca. Garantire la compatibilità retroattiva e l'interoperabilità durante il periodo di transizione è fondamentale per mantenere fiducia e sicurezza nella rete.

Applicazioni e servizi IT

Anche in quest'ambito la migrazione verso soluzioni resistenti richiede l'aggiornamento delle implementazioni crittografiche sottostanti, l'adattamento ai cambiamenti nelle dimensioni delle chiavi e nelle prestazioni degli algoritmi e l'assicurazione della compatibilità con protocolli e librerie aggiornati. Gli sviluppatori devono pertanto: modificare il codice, condurre test approfonditi e se necessario riprogettare le interfacce utente per far fronte a questi cambiamenti.

3.3.5 Approcci ibridi come strategia di transizione

Come accennato in precedenza in questa sezione vengono presentate alcune alternative di utilizzo in forma ibrida della PQC[12].

Firma digitale

Le tecniche comuni per le firme digitali ibride prevedono l'uso di firme doppie, che consistono in due o più firme su un messaggio comune. Possono essere conosciute anche come firma ibrida o firma composita. La verifica della firma doppia richiede che tutte le firme componenti siano verificate con successo, ad esempio creando una singola firma composita logica a partire da due o più algoritmi di firma componenti. Le firme doppie potrebbero essere utilizzate per firmare dati dell'utente (ad esempio un documento o un'e-mail) o certificati digitali che contengono riferimenti alle coppie di chiavi dell'utente all'interno di una PKI. Gli standard e le linee guida esistenti del NIST ne consentono l'uso a condizione che almeno un algoritmo di firma digitale componente sia stato approvato.

Key encapsulation

Un KEM ibrido viene definito come una combinazione di due o più componenti KEM che diventa quindi un metodo composito di due o più meccanismi di scambio e incapsulamento delle chiavi. La proprietà desiderata dalle tecniche ibride è che le chiavi derivate rimangano sicure se almeno uno degli schemi componenti è sicuro. Inoltre, l'output dello schema di stabilimento della chiave è una chiave segreta condivisa, un segreto condiviso che non richiede ulteriori derivazioni della chiave.

Crittografia simmetrica

Non sono state considerate alternative ibride dal NIST in merito a questo tipo di algoritmi perché gli standard già presenti, con dimensioni delle chiavi adeguate, sono considerati resistenti agli attacchi dei computer quantistici. Il principale tipo di attacco quantistico su crittosistemi ibridi è l'**Algoritmo di Grover**.

Capitolo 4

Problematiche e Sfide nella Migrazione verso il Post-Quantum

In questo capitolo analizziamo le problematiche attuali dei quantum computer e dei qubit, le difficoltà di transizione verso algoritmi e componenti di PQC, le tipologie note di attacchi post-quantum e l'impatto che hanno i sistemi quantum-based sulle infrastrutture e sui sistemi legacy¹ e IoT².

4.1 Lo stato dell'arte dell'hardware quantistico: l'era NISQ e il percorso verso la fault-tolerance

Stando al documento[10], la tecnologia quantistica sta attraversando una fase di transizione critica, spostandosi dalla pura era NISQ verso la prima fase di correzione degli errori attiva. Secondo la classificazione proposta da John Preskill, l'obiettivo immediato è il raggiungimento del regime **Megaquop** (Mega-Quantum-Operations), caratterizzato dalla capacità di eseguire circa 10^6 operazioni logiche coerenti prima che la probabilità di errore comprometta il risultato. Per abilitare tale capacità computazionale, è necessario implementare codici di correzione con una distanza del codice sufficientemente elevata.

Distance code e qubit logici

Nel computer quantistico un singolo qubit risulta essere troppo rumoroso. Per risolvere il problema si raggruppano i qubit fisici per creare un singolo **qubit logico**, questo 'gioco di squadra' è dettato da alcuni codici di correzione, il più famoso è il *Surface Code* all'interno del quale la code distance viene utilizzata come unità di misura della tolleranza ai guasti. Il numero

¹Si definisce legacy 'dal termine inglese eredità' l'insieme di sistemi o componenti informatici datati rispetto agli standard e ai protocolli in uso ma non per questo abbandonati

²Connessione di oggetti fisici a Internet, con un'identità digitale e la capacità di scambiare dati tra loro e con sistemi esterni, senza intervento umano diretto (es: alcuni sensori)

d (distanza del codice) indica quanto è grande e robusta la griglia di bit logici. Un codice con distanza d può correggere $\frac{d-1}{2}$ errori. Dunque la 'Code Distance' è una metrica che misura la capacità di un qubit logico di sopprimere gli errori: all'aumentare della distanza (3, 5, 7...), la probabilità di un errore logico diminuisce esponenzialmente, al costo di un drastico aumento del numero di qubit fisici necessari. Le dimostrazioni sperimentali più recenti (Google Quantum AI, IBM) hanno validato con successo il passaggio da una distanza del codice $d = 3$ (circa 17 qubit fisici per uno logico) a $d = 5$ (circa 49 qubit fisici per uno logico). Questo salto qualitativo è fondamentale: mentre a $d = 3$ la correzione è minima, a $d = 5$ la soppressione esponenziale degli errori diventa evidente permettendo ai qubit logici di sopravvivere abbastanza a lungo per sostenere la complessità algoritmica richiesta dall'era Megaquop.

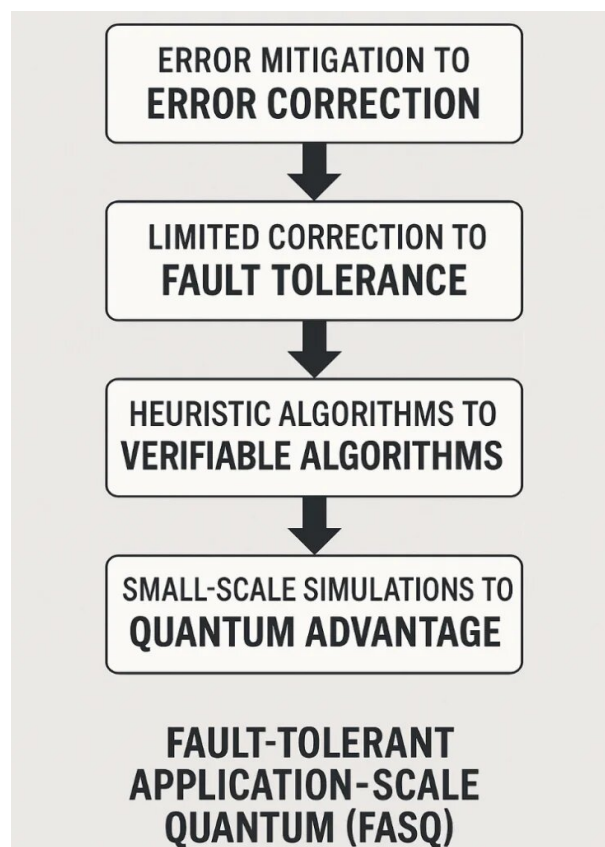


Figura 4.1: Prossimi stadi operativi in ambito QEC (Quantum Error Correction)

Abbiamo dunque appena varcato la soglia di questa nuova era dove la ridondanza dei qubit fisici inizia a generare un vantaggio computazionale netto rispetto ai singoli componenti fisici, inoltre i tassi di errore sui gate a due qubit ora si avvicinano allo 0,1% sui migliori dispositivi superconduttori. Gli errori di misurazione e sul singolo qubit sono scesi sotto l'1%. Questi risultati suggeriscono che la fisica fondamentale dei qubit è sempre più ben compresa e che i laboratori possono eseguire operazioni quantistiche su scala significativa. Tra qualche anno i sistemi potrebbero raggiungere la soglia in cui alcune simulazioni scientifiche e compiti di ottimizzazione effettuati diverranno impossibili da eseguire in tempo ragionevole per i computer

classici. È importante sottolineare che i ricercatori sostengono che nessuna singola piattaforma hardware ha un vantaggio chiaro. I circuiti superconduttori offrono velocità e maturità di fabbricazione, mentre i sistemi atomici offrono lunghi tempi di coerenza e alta connettività. Lo studio dei ricercatori inoltre dedica la stessa importanza ad hardware e software, in particolare, alla necessità di metodi rigorosamente definiti con accelerazioni misurabili. L'articolo[10] indica che la competizione tra i team di simulazione quantistica e classica rimane accesa. Questo perché risultati un tempo considerati traguardi quantistici sono stati rapidamente eguagliati da algoritmi classici migliorati. Tale dinamica tuttavia si dimostra sana e continuerà a spingere entrambe le comunità fino a che il vantaggio quantistico non diventerà innegabile. Lo sviluppo quantistico avverrà quindi in fasi (megaquop, gigaquop, teraquop, eccetera) e gli utilizzi più importanti che esso potrà avere non saranno predicibili del tutto finché non saranno disponibili tecnologie completamente fault-tolerant[13].

4.2 Complessità degli algoritmi PQC

Gli algoritmi post-quantistici³ sono noti per essere ben più complessi di quelli adottati precedentemente soprattutto perché le operazioni matematiche sottostanti non si basano più sulla 'semplice' aritmetica modulare ma su operazioni vettoriali e matriciali complesse o sull'uso intensivo di codici di correzione degli errori. Questo comporta un aumento dei cicli di CPU necessari per l'incapsulamento delle chiavi e la generazione delle firme digitali. Gli algoritmi sono difficili da progettare: sia per chi attacca utilizzando un quantum-computer sia per chi si difende da un attacco.

- Dal punto di vista dell'attaccante: gli algoritmi quantum-based sono difficili da progettare perché ciò significa manipolare uno stato in cui tutte le possibili risposte esistono contemporaneamente. Non si tratta di 'provare tutte le strade una per una velocemente', ma di calcolarle tutte insieme in parallelo. Gestire mentalmente e matematicamente questa molteplicità è estremamente complesso. Inoltre, quando si misura lo stato in un algoritmo quantistico per determinarne il risultato, tutte le informazioni precedenti alla misurazione vengono perse, ciò avviene a causa della riduzione del qubit a un solo stato (collasso) che impedisce di misurare a 'runtime' come sta performing l'algoritmo. Non ci è nemmeno permesso effettuare la copia di un valore, questo perché non è possibile creare una copia di uno stato quantistico sconosciuto. Un'ulteriore limitazione è costituita dalla non reversibilità delle porte logiche classiche utilizzate in algebra booleana.
- Dal punto di vista del difensore: gli algoritmi di difesa da attacchi quantum richiedono chiavi molto lunghe, risultando anche in una difficoltà di trasmissione in rete delle chiavi stesse o delle firme digitali. Per non parlare delle difficoltà matematiche e implementative

³Si intendono gli algoritmi PQC utilizzati sui computer classici per proteggerci dalla minaccia quantistica

di tali algoritmi già discusse nelle sezioni precedenti, motivo per il quale spesso si ricorre ad algoritmi ibridi aumentando però il costo computazionale.

4.2.1 Complessità matematica

Tuttavia, se si analizza la complessità computazionale asintotica, lo scenario si inverte a favore degli algoritmi post-quantistici su reticoli. I sistemi a chiave pubblica classici, come RSA, richiedono operazioni di esponenziazione modulare che comportano una complessità tipicamente cubica, $O(k^3)$, rispetto alla lunghezza della chiave k . Anche la moltiplicazione scalare su curve ellittiche (ECC), pur operando su chiavi più corte, mantiene un costo computazionale elevato per bit di sicurezza. Al contrario, gli schemi basati su reticoli strutturati (es. Module-LWE per ML-KEM) operano su anelli di polinomi. Grazie all'implementazione della Number Theoretic Transform (NTT) (una variante della Fast Fourier Transform (FFT) per campi finiti) la moltiplicazione tra polinomi, che rappresenta l'operazione più onerosa, può essere eseguita con una complessità di $O(n \log n)$, dove n è il grado del polinomio.

4.2.2 Impatto computazionale su CPU, memoria e rete

Di conseguenza come discusso nel report del NIST[1], il 'collo di bottiglia' computazionale della PQC non risiede nella complessità matematica dell'algoritmo in sé (che è asintoticamente molto efficiente), quanto piuttosto nel costo dell'accesso alla memoria (memory-bound) e nella gestione di grandi moli di dati (I/O-bound) poiché gli schemi PQC (in particolare quelli basati su reticoli, come Kyber/ML-KEM e Dilithium/ML-DSA) richiedono chiavi e firme nell'ordine dei kilobyte. Questo aumento di ordini di grandezza introduce problemi di latenza nelle comunicazioni di rete (causando la frammentazione dei pacchetti IP e aumentando i tempi di handshake TLS)[14]. Le operazioni primitive della PQC inoltre si basano su algebra lineare complessa che, in assenza di acceleratori hardware dedicati nelle CPU attuali, comporta un maggior numero di cicli di clock e, conseguentemente, un maggiore consumo energetico.

4.3 Sfide architetturali e di integrazione

Oltre alla complessità legata alla dimensione dei dati, l'integrazione della PQC altera fondamentalmente l'architettura di un sistema. Il ruolo della **Crypto-Agility** richiede un'astrazione architetturale totale. In un'architettura agile, le applicazioni devono usare API generiche delegate a un provider crittografico sottostante. Molte aziende non sanno dove e quali algoritmi usano nel loro software, dunque la sfida si traduce nell'implementazione di una CBOM (Cryptography Bill of Materials), un vero e proprio inventario composto da API crittografiche [15][16]. Architetturealmente, i sistemi di CI/CD (Continuous Integration) devono essere modificati per generare dinamicamente una mappa di tutte le dipendenze crittografiche del sistema, permettendo agli amministratori di orchestrare una migrazione mirata senza rompere le dipendenze software.

Inoltre, alcuni algoritmi approvati dal NIST sono Stateful Hash-Based. La loro sicurezza si basa su un albero di Merkle (One-Time Signatures) in cui ogni chiave privata può essere usata una e una sola volta. Architetturealmente, il sistema deve mantenere un contatore (stato) persistente. La sfida è immensa nelle moderne architetture Cloud e nei sistemi distribuiti:

- **Clonazione e Snapshot:** Se una Macchina Virtuale (VM) o un container (es. Docker) fa un backup, esegue una firma (aggiornando lo stato), e poi viene ripristinata al backup precedente (per cause spesso accidentali o per malfunzionamenti), lo stato torna indietro. Alla firma successiva, la chiave monouso verrà riutilizzata, compromettendo totalmente il sistema;
- **Load Balancing:** Se due server cluster condividono la stessa chiave privata stateful, sincronizzare il contatore tra loro in tempo reale senza incappare in race conditions (dove entrambi usano lo stesso indice contemporaneamente) diviene un problema architetturale particolarmente rilevante.

Un altro problema architetturale spesso ignorato è l'elevata necessità di entropia degli algoritmi PQC: gli algoritmi basati su reticoli (Lattice-based, ad esempio ML-KEM) richiedono l'uso estensivo di funzioni hash (SHAKE) e una quantità massiccia di bit casuali di alta qualità (selezionati attraverso Rejection Sampling⁴, in modo tale da rispettare i rigidi principi matematici dei reticoli) per la generazione delle matrici e del rumore. In architetture con generatori di numeri casuali hardware (TRNG) lenti, il sistema rischia la Entropy Starvation⁵, causando il blocco temporaneo dell'architettura in attesa di nuova casualità, aumentando drasticamente i tempi di latenza del server[17]. Questa serie di problemi è risolvibile utilizzando PRNG (Pseudorandom Number Generators) crittograficamente sicuri e accelerati (es: SHAKE-based DRBG o AES-CTR DRBG) che invece di utilizzare entropia pura (TRNG) per ogni singola matrice del reticolo, usano il TRNG solo per generare un singolo 'seed' di 256 bit di pura entropia (operazione lenta). Questo seme viene poi dato in pasto a un chip hardware AES o SHA-3 che lo espande, generando gigabyte di numeri pseudo-casuali a velocità altissime. In questo modo se il seme iniziale è totalmente ignoto, l'espansione è sicura[16]. Per il Cloud invece, nelle architetture SOTA⁶, il server supervisore concreto che ha un TRNG hardware (Hypervisor), espone un canale diretto e sicuro chiamato *virtio-rng* alle macchine virtuali. In questo modo, il kernel della Virtual Machine utilizza l'entropia pura direttamente dalla scheda madre del server fisico, prevenendo la starvation al momento del boot.

⁴Metodo di campionamento statistico che prevede lo scarto di campioni non conformi ad un dato criterio arbitrario

⁵Esaurimento dell'entropia, condizione che comporta una perdita di casualità tale da rendere l'output dei generatori crittografici prevedibile, compromettendo la sicurezza del sistema e la robustezza delle chiavi generate

⁶Attuali, allo stato dell'arte, (State Of The Art)

4.3.1 Impatto su sistemi legacy e dispositivi IoT (risorse limitate)

Ancora peggiore è l'impatto di questo tipo di algoritmi quando vengono adoperati su sistemi legacy, embedded o IoT (sensori, smart grid, dispositivi medici) che operano in ambienti 'constrained' (con risorse limitate) e che spesso sono dotati soltanto di pochi kB di RAM, portandoli molto spesso ad inevitabili problematiche di *overflow dello stack* di memoria. Un esempio è stato prodotto utilizzando il microcontrollore IoT (ARM Cortex-M4, che il NIST ha usato come benchmark[18]) che dispone solo di poche decine di Kilobyte di BRAM⁷. Caricare in memoria una chiave pubblica PQC e lo stato interno dell'algoritmo può esaurire completamente la RAM disponibile, impedendo al dispositivo di svolgere la sua funzione primaria. Questo non è l'unico problema però, poiché nell'IoT si usano spesso reti a bassa potenza (LPWAN, LoRaWAN, Bluetooth Low Energy). Trasmettere un handshake TLS con certificati PQC (che pesano svariati Kilobyte) richiede l'invio di decine di pacchetti radio in più rispetto al passato. Poiché la trasmissione radio è l'operazione che consuma più batteria in un sensore IoT, l'uso della PQC rischia di dimezzare o peggio la vita operativa delle batterie dei dispositivi remoti. Dunque spesso c'è grande necessità di frammentazione dei pacchetti IP con il rischio di saturare il canale radio, è sufficiente perdere anche uno soltanto dei frammenti per invalidare l'intero messaggio[19]. I chip IoT inoltre necessitano di HSM (hardware crittografico) dedicato per svolgere la PQC in hardware, è quindi di fondamentale importanza garantire che i moduli hardware consentano di eseguire operazioni di PQC in modo efficiente. Molti sistemi legacy o vecchi dispositivi IoT non possiedono meccanismi sicuri per essere aggiornati da remoto, oppure sono installati in luoghi inaccessibili (es. satelliti, sensori sottomarini, dispositivi medici impiantati). Per questi sistemi, **la migrazione verso la PQC è impossibile senza una sostituzione fisica dell'hardware**. È doveroso considerare il ruolo critico delle infrastrutture legacy all'interno della migrazione, che costituiscono una porzione preponderante dell'attuale ecosistema tecnologico: secondo le recenti stime sulla preparazione Post-Quantum, tali sistemi rappresentano circa il 70% delle infrastrutture operative crittograficamente impreparate[15].

4.4 Nuovi vettori di attacco

Nell'analizzare il panorama delle minacce introdotte dal calcolo quantistico, è fondamentale premettere un principio di inclusività computazionale: un computer quantistico universale è in grado di simulare, seppur con diversa efficienza, qualsiasi operazione logica eseguibile da una macchina di Turing classica. Di conseguenza, l'intero spettro delle metodologie di attacco convenzionali (dalle vulnerabilità software alle tecniche di ingegneria sociale) rimane teoricamente applicabile e valido anche in un'era post-quantistica. Tuttavia, la vera criticità non risiede nella mera replicazione degli attacchi noti, bensì nell'emergere di vettori di minaccia qualitativamente nuovi. L'architettura quantistica abilita primitive crittanalitiche impossibili per

⁷Block RAM, una tipologia di RAM spesso usata in sistemi IoT programmabili

l'hardware classico (come l'algoritmo di Shor per la fattorizzazione o l'algoritmo di Grover per la ricerca non strutturata), ridefinendo il concetto stesso di 'problema intrattabile'. Oltre alla minaccia algoritmica diretta, si delineano nuove strategie offensive che sfruttano le vulnerabilità temporali della migrazione e le complessità fisiche delle nuove implementazioni crittografiche. Nelle sezioni seguenti verranno analizzate due delle minacce più insidiose per la sicurezza dei dati nell'attuale fase di transizione verso il post-quantum: la strategia di intercettazione preventiva (Store Now, Decrypt Later) e le vulnerabilità fisiche legate all'esecuzione degli algoritmi (Side-channel).

4.4.1 L'attacco 'Store Now, Decrypt Later' (SNDL)

L'attacco denominato Store Now, Decrypt Later (SNDL), noto anche come Harvest Now, Decrypt Later, rappresenta una minaccia strategica e retroattiva che inverte il tradizionale paradigma di sicurezza informatica che mira a violare un sistema in tempo reale. L'SNDL sfrutta il distacco temporale tra la cattura del dato e la capacità tecnologica futura di decifrarlo. Sebbene i dati siano attualmente illeggibili poiché protetti da algoritmi classici robusti, essi diventeranno vulnerabili non appena l'attaccante avrà accesso a un computer quantistico crittograficamente rilevante (CRQC). Utilizzando uno degli algoritmi trattati (**Shor** e **Grover**) l'avversario potrà derivare le chiavi private utilizzate al momento della cattura e decifrare retroattivamente tutto lo storico delle comunicazioni archiviate. La criticità dell'SNDL dipende dalla 'vita utile' del segreto (shelf-life):

- Per dati effimeri (es. una transazione web banale), la minaccia è trascurabile.
- Per dati sensibili a lungo termine (es. segreti di stato, cartelle cliniche, dati genomici, identità di agenti sotto copertura), la minaccia è immediata.

Questa minaccia rende inefficace il concetto di Perfect Forward Secrecy (PFS) basato su curve ellittiche classiche, poiché è lo stesso scambio di chiavi iniziale a essere matematicamente vulnerabile. L'unica mitigazione possibile è l'adozione immediata di algoritmi PQC o ibridi per lo scambio di chiavi, proteggendo il traffico attuale dalla decifrazione futura.

4.4.2 Vulnerabilità side-channel nelle implementazioni PQC

L'attacco side-channel si basa sulla vulnerabilità dell'implementazione fisica, sui sistemi hardware, dei problemi complessi di crittografia. Esso non mira a rompere la matematica dietro l'algoritmo, ma ad estrarre informazioni segrete utili (come ad esempio il consumo e le emissioni fisiche del dispositivo) durante l'esecuzione dei calcoli. Per prevenire Timing Attacks (attacchi basati sulla misurazione del tempo di esecuzione) un algoritmo crittografico deve impiegare esattamente lo stesso numero di cicli di clock indipendentemente dai dati processati (chiavi o messaggi). Tuttavia, come precedentemente accennato, molti algoritmi PQC (inclusi alcuni basati su reticoli) utilizzano tecniche probabilistiche come il campionamento con rigetto (rejection

sampling) per generare vettori con specifiche distribuzioni di probabilità. Poiché il numero di 'rigetti' (ripetizioni) varia in modo casuale, il tempo di esecuzione non è costante. Forzare questi algoritmi a operare in tempo costante richiede artifici implementativi complessi che, se non realizzati perfettamente, lasciano trapelare informazioni sulla chiave segreta attraverso le variazioni temporali. Gli attacchi basati sull'analisi differenziale della potenza (DPA) o sulle emissioni elettromagnetiche sfruttano il fatto che manipolare un bit di valore '0' consuma una quantità di energia diversa rispetto a un '1'. Data la grandezza delle chiavi e la complessità delle operazioni polinomiali negli schemi PQC, la quantità di operazioni di memoria e aritmetiche è molto elevata. Questo genera un segnale (traccia di consumo) molto ricco e lungo, offrendo all'attaccante più punti di dati statistici per correlare il consumo energetico ai valori della chiave segreta. Questo tipo di attacchi riguarda soprattutto Smart Card IoT e dispositivi che l'attaccante può avere fisicamente in mano. Su un server cloud protetto, ad esempio, il Side-Channel è meno probabile (anche se esistono attacchi remoti via rete/cache).

Capitolo 5

Attività Pratica: Implementazione di un Prototipo di Migrazione PQC

In questo capitolo è stato documentato un esempio pratico di implementazione, da parte dell'autore, di PQC ad approccio ibrido che sfrutta due meccanismi di incapsulamento e scambio delle chiavi già esistenti in letteratura.

5.1 Contesto e piano di lavoro: il progetto AstroVerse

L'obiettivo del lavoro è stato quello di operare un meccanismo di incapsulamento delle chiavi basato su PQC ibrida e *crittografia a riposo*¹, utilizzando come algoritmo di cifratura classico ECDH per il primo segreto generato e ML-KEM (standard SOTA NIST per la PQC) per il secondo segreto generato, andando così a costituire, attraverso la combinazione delle due chiavi, una Key Derivation Function (KDF). Questo meccanismo di incapsulamento verrà implementato tramite un Servizio di Crittografia Ibrida all'interno del branch 'quantum' di una repository già esistente di proprietà dell'autore, riguardante una piattaforma social web open-source, di nome AstroVerse. L'algoritmo implementato avrà lo scopo di cifrare i dati sensibili dell'utente. Il progetto in realtà gestisce già in maniera efficace l'hashing delle password utilizzando una classe di algoritmi della libreria BCrypt (BCryptPasswordEncoder), la cui funzione hash garantisce già una certa resistenza a tipologie di attacchi post-quantum, stando ai livelli di sicurezza del NIST [4]. Questo meccanismo sarà dunque utilizzato per garantire una protezione standardizzata dal NIST all'email degli utenti di AstroVerse. Verrà inoltre gestita la migrazione all'algoritmo per gli utenti già registrati con un email non protetta da PQC, mediante l'utilizzo di un componente Java. Ulteriori informazioni sono disponibili sulla [repository ufficiale di AstroVerse, branch quantum](#), i file antecedenti alle modifiche PQC sono invece presenti sul branch main.

¹L'*encryption at rest*, nota anche come crittografia dei dati a riposo, è una pratica di sicurezza informatica che prevede la crittografia dei dati mentre sono archiviati su dispositivi di archiviazione fisici o digitali come dischi rigidi, database, archiviazione cloud o unità USB

5.2 Scelte tecnologiche

Il server back-end è stato sviluppato interamente attraverso il framework di Java 'Spring Boot', dunque la libreria Java standard de facto, Bouncycastle ci sarà utile per preparare l'ambiente. Il front-end invece è stato sviluppato utilizzando il framework di Javascript 'Vue.js'. Procediamo dunque inserendo le dipendenze Maven aggiornate allo standard FIPS 203[1] finalizzato dal NIST, migrando da Kyber (draft) a ML-KEM (standard). All'interno del progetto quindi, sono state aggiunte le dipendenze di Bouncycastle alle proprietà già presenti nel file 'pom.xml' (**Appendice A1**).

5.3 Progettazione del servizio di crittografia ibrida (HybridCryptoService)

La classe java service sarà il cuore pulsante del nuovo crittosistema implementato su Astroverse. Per prima cosa inseriamo il nuovo security provider, BouncyCastle, poi generiamo 2 coppie di chiavi: una per la PQC e una per il sistema Diffie-Hellmann basato su curve ellittiche. Per fare ciò ci serviamo di un generatore di chiavi deterministico basato su seed in modo tale che anche se un utente effettua la registrazione e il server viene riavviato le sue chiavi risulteranno comunque stabili tra i riavvii. Si noti che l'utilizzo di un seed statico incorporato nel codice è funzionale all'ambiente di dimostrazione del tirocinio; in un ambiente di produzione reale, la persistenza delle chiavi master verrebbe delegata a un Hardware Security Module (HSM) o a un Key Management System sicuro (**Appendice A2**).

5.3.1 Generazione delle chiavi (ECC e ML-KEM)

Fatto ciò utilizziamo una classe contenitore per mantenere i dati cifrati, la tipologia di incapsulamento e la chiave pubblica del client e andiamo a definire i metodi per la cifratura e decifratura insieme ai vari componenti necessari per farli funzionare, quali: **hashString** che crea un hash come indice per cercare la chiave cifrata dell'utente che altrimenti sarebbe illeggibile come testo in chiaro; **combineSecrets** che unirà la chiave Elliptic Curve con quella Lattice Based (PQC).

5.3.2 Processo di cifratura ibrida (ECDH + ML-KEM + AES-GCM)

Adesso definiamo le funzioni **aesEncrypt** e **aesDecrypt**, componenti rispettivamente per la cifratura e decifratura della chiave finale utilizzando l'algoritmo AES in modalità Galois/Counter Mode, che ci consente di garantire confidenzialità, autenticità e integrità del segreto. Successivamente sviluppiamo la funzione dove avverrà integralmente il processo che cifrerà il testo in chiaro (email utente). Procediamo dunque alla creazione del record contenitore per i dati cifrati e alla **fase di cifratura**. Questa fase consisterà nel simulare un client ECC che genera la

coppia di chiavi a curva ellittica (il termine simulare intende che normalmente si agisce da client remoto che chiede al server la coppia di chiavi EC generate, dunque agiamo come se il client fosse separato, anche se gira in locale). Restituiamo attraverso l'agreement simulato del server, la chiave privata (relativa alla cifratura classica). Generiamo ora la chiave privata incapsulata attraverso ML-KEM e richiamando la funzione `combineSecrets` andiamo a combinare i due segreti nella chiave finale. Non ci resta che cifrare tramite la funzione `aesEncrypt` il nostro testo in chiaro (email utente AstroVerse) con la chiave finale risultante dalla combinazione delle due private generate ECDH e ML-KEM e il gioco è fatto.

5.3.3 Processo di decifratura

Procediamo ora con la fase di **decifratura ibrida** composta dagli stessi passi della fase di cifratura, a differenza del fatto che la modalità con cui verranno utilizzati i segreti (le due chiavi private ECDH e ML-KEM) sarà di decifratura. Inoltre verrà utilizzata la funzione `aesDecrypt` precedentemente descritta.

5.4 Modifiche al back-end per supportare la PQC

Andiamo ora a modificare la classe `UserService` in modo da poter utilizzare in essa l'email cifrata. Implementiamo inoltre un metodo che ci consente di vederla in chiaro nella visualizzazione account (**Appendice A9**). Il componente `User Data Transfer Object` verrà modificato per contenere il campo `isQuantumEncrypted` affinché sia possibile verificare attraverso richiesta se la mail dell'utente in questione è protetta da PQC (**Appendice A10**), questo campo ci servirà durante lo sviluppo del componente per la migrazione automatica degli account legacy².

5.4.1 Adeguamento del modello dati (Entity User) e della persistenza

Una volta terminato il core del progetto, andiamo a modificare alcuni campi dell'entità persistente `user` al fine di adeguare tutti i dati dell'utente alla migrazione (**Appendice A11**). Le modifiche effettuate renderanno così persistenti i dati che andremo ad inserire: l'hash che funge da indice per l'incapsulamento delle chiavi, la chiave pubblica e l'intera chiave incapsulata. Andiamo poi ad aggiungere i metodi JPA necessari per le funzioni che andremo a sviluppare nel componente per la migrazione (**Appendice A12**).

²Si intende per account legacy nel contesto di sviluppo di questo progetto, il tipo di account al quale non è stata ancora applicata l'adeguata protezione PQC

5.4.2 Integrazione del servizio ibrido nel flusso di registrazione e di modifica utente

Modifichiamo i metodi del controller in modo da salvare la mail in chiaro nel metodo della registrazione prima di cifrarla, per garantirne il corretto inserimento all'interno del jwt token (**Appendice A13**). La lasciamo invariata invece nel login poiché viene decifrata attraverso il metodo `userService.getUser()`, inoltre nel metodo `viewAccount` passiamo al frontend attraverso il DTO anche il campo `isQuantumEncrypted` in modo da poterne validare l'utilizzo nel frontend.

5.4.3 Implementazione di un componente per la migrazione automatica degli utenti legacy

Ora procediamo alla creazione di un nuovo componente Java che effettua la migrazione degli indirizzi email con il campo `'isQuantumEncrypted' = 0`, in modo tale da gestire anche gli account legacy di AstroVerse. L'event listener `ApplicationReadyEvent.class` ci garantisce che questo componente viene eseguito ogni volta che il backend server viene avviato (**Appendice A14**). Il metodo di migrazione applicato sarà lo stesso utilizzato precedentemente nel processo di cifratura ibrida.

5.5 Modifiche al front-end (Vue.js) per il feedback utente

Per completare il ciclo di migrazione, è necessario adeguare il client frontend (sviluppato in framework Vue.js) affinché sia in grado di interpretare lo stato di sicurezza dell'account e comunicarlo all'utente. Poiché la cifratura *at-rest* è trasparente all'utente finale, senza un feedback visivo non vi sarebbe percezione dell'avvenuto aggiornamento di sicurezza. Lato client, all'interno del ciclo di vita del componente principale (`Home.vue`), è stata implementata una logica di controllo nell'hook³ `onMounted`. Per evitare di degradare l'esperienza utente mostrando la notifica ad ogni caricamento della pagina, è stato utilizzato il `sessionStorage` del browser. Questo garantisce che il feedback visivo (Toast Notification) appaia una sola volta per sessione di navigazione, confermando all'utente l'utilizzo della crittografia ibrida Post-Quantum per la protezione dell'email (**Appendice A15**).

5.6 Risultati e considerazioni sull'implementazione pratica

Con quest'ultima modifica il progetto è ultimato e pronto a cifrare in maniera sicura le email sia degli utenti già registrati (legacy users) sia dei nuovi utenti. Come abbiamo potuto notare il processo è risultato semplice ed efficace se consideriamo l'importanza e l'impatto che deriva dal garantire questo tipo di protezione e dall'agire pro-tempore in vista dell'avvento dei CRQCs.

³Tipologia di funzione Javascript basata su eventi (event-driven)

È da notare inoltre che la gestione singola del solo algoritmo di PQC avrebbe richiesto un costo computazionale nettamente inferiore e pertanto sarebbe stato ancora più semplice implementarlo singolarmente, a dimostrazione empirica del fatto che la PQC ibrida aumenta il costo computazionale.

Capitolo 6

Conclusioni e Sviluppi Futuri

In questo capitolo verranno infine presentate le ultime considerazioni dell'autore, apprese attraverso lo studio dei risultati ottenuti in letteratura allo stato dell'arte e attraverso l'implementazione pratica della PQC ibrida.

6.1 Considerazioni finali sullo stato della migrazione e l'urgenza di agire

Dai risultati ottenuti in ricerca possiamo stimare che l'era dei computer quantistici completamente Fault-Tolerant è ancora lontana nonostante i notevoli e recenti progressi nel campo. Abbiamo visto come, allo stato attuale, ciò sia dovuto principalmente ai limiti dell'hardware oggi disponibile: numero di qubit ancora insufficiente, elevata rumorosità, errori sui gate e in lettura, tempi di coerenza limitati e assenza di una correzione d'errore quantistica pienamente scalabile. Di conseguenza, risulta complesso mantenere la coerenza quantistica per la profondità circuitale necessaria ad attaccare con successo informazioni protette. Alla luce di quanto esposto, lo studio dettagliato del quantum, così come delle problematiche da esso derivanti, non dovrebbe assolutamente scoraggiare. È bene ricordare però che l'ambito quantistico abbraccia un'ampia gamma di discipline e tematiche, il cui approfondimento può risultare complesso e, senza dubbio, non banale. Si raccomanda pertanto uno studio dettagliato e approfondito degli ambiti matematici e fisici, di notevole impatto teorico e applicativo nel campo della fisica e della computazione quantistica. La tipologia di attacchi SNDL resta la forma di pericolo più imminente e risulta di vitale importanza prevenirla in tempo utile.

6.2 Sensibilizzazione all'adozione di standard PQC e approcci ibridi

L'autore invita pertanto a prendere parte attivamente alla fase di migrazione verso le nuove tecnologie di difesa e a non sottovalutare i rischi connessi alle minacce emergenti, soprattutto considerando che il processo di migrazione potrebbe protrarsi per un periodo significativo. Come illustrato anche a seguito dell'attività pratica è necessario sensibilizzare l'utilizzo di nuove tecnologie PQC, anche semplicemente in forma ibrida, così da preservare la riservatezza delle informazioni sensibili anche in presenza di un eventuale indebolimento futuro degli algoritmi classici, facendo leva sulla componente post-quantum della soluzione (e viceversa). Di vitale importanza è anche la gestione di eventuali caratteristiche legacy (come nel caso del progetto pratico, gli utenti registrati precedentemente alla patch PQC), sicuramente presenti in grande quantità e direttamente proporzionali alla vastità di utilizzo di un sistema software.

6.3 Difficoltà incontrate: il 'Quantum Hype' e le limitazioni dell'hardware attuale

La vastità del panorama del *quantum computing*, che interseca domini eterogenei quali la crittografia, l'intelligenza artificiale e la fisica, ha rappresentato una sfida significativa nella stesura di questo elaborato. La principale difficoltà è consistita nel discernere le reali capacità tecnologiche attuali dalle aspettative inflazionate, un fenomeno noto in letteratura come *Quantum Hype*. Diverse analisi di settore, tra cui quelle riportate nel *Gartner Hype Cycle*, evidenziano come le tecnologie quantistiche si trovino spesso in una fase di aspettative sovrastimate rispetto alla loro immediata applicabilità industriale [20], [21]. È stato dunque prioritario garantire l'attendibilità delle informazioni, filtrando le notizie sensazionalistiche per concentrarsi sulle tecniche verificate e sugli standard definiti dagli enti regolatori (NIST). Con la futura maturazione di hardware quantistico crittograficamente rilevante (*Cryptographically Relevant Quantum Computers* - CRQCs), questo documento necessiterà inevitabilmente di aggiornamenti. In uno sviluppo futuro del progetto, sarebbe di grande interesse trattare un caso reale di migrazione di un'infrastruttura *legacy* aziendale, analizzando le metriche di performance degli algoritmi PQC (come ML-KEM) in ambienti di produzione ad alto traffico. L'analisi sperimentale offensiva (tentativo di decifratura di un algoritmo classico) è stata considerata ma risulta purtroppo non attuabile nel presente lavoro a causa delle limitazioni dell'hardware attuale: le piattaforme cloud accessibili (come IBM Quantum o Google Quantum AI) operano ancora in regime NISQ (*Noisy Intermediate-Scale Quantum*). Attualmente, il numero di qubit fisici e, soprattutto, la mancanza di qubit logici corretti dagli errori, rendono impossibile l'esecuzione dell'algoritmo di Shor su chiavi di lunghezza commerciale. Si stima infatti che per rompere una chiave RSA-2048 siano

necessari circa 20 milioni di qubit fisici rumorosi [22], un ordine di grandezza ben lontano dalle attuali centinaia o migliaia di qubit disponibili [13].

6.4 Prospettive future: dall'analisi sperimentale offensiva al Quantum Machine Learning

Questo elaborato è stato quindi incentrato sulla difesa crittografica, poiché per evidenti problematiche riscontrate con l'hardware attualmente disponibile non sarebbe stato, in alcun modo possibile effettuare analisi sperimentale offensiva o penetration testing. Non vanno trascurate, tuttavia, le importanti implicazioni di possibili sviluppi in entrambe le aree tematiche. Inoltre un'altra branca il cui sviluppo sembra procedere molto più velocemente rispetto alla migrazione verso la PQC è quella del calcolo quantistico nel campo del Machine Learning (*Quantum Machine Learning* o *Quantum Enhanced Machine Learning*¹), un ambito che promette di rivoluzionare l'analisi dei dati[23] e che meriterebbe una trattazione dedicata in lavori successivi.

¹Il settore del Machine Learning che sfrutta la potenza di calcolo dei Quantum Computers, nello specifico dei qubit stabili per processare dati e con essi addestrare grandi modelli di Machine Learning (LLMs, Large Language Models)

Riferimenti bibliografici

- [1] National Institute of Standards and Technology, «Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203),» U.S. Department of Commerce, FIPS PUB 203, ago. 2024, Descrive l'uso della Number Theoretic Transform (NTT) per ottimizzare la moltiplicazione polinomiale in ML-KEM. indirizzo: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] D. Milmo, «Google hails breakthrough as quantum computer surpasses ability of supercomputers,» *The Guardian*, ott. 2025, Accessed: 2025-11-12. indirizzo: <https://www.theguardian.com/technology/2025/oct/22/google-hails-breakthrough-as-quantum-computer-surpasses-ability-of-supercomputers>
- [3] M. Mosca e M. Piani, «Quantum Threat Timeline Report 2024,» Global Risk Institute, Annual Report, 2024, Sondaggio tra esperti globali: la maggioranza ritiene bassa la probabilità che la crittografia RSA-2048 venga violata entro 10 anni, spostando il rischio elevato verso la metà degli anni '30. indirizzo: <https://globalriskinstitute.org/publications/quantum-threat-timeline-report-2024/>
- [4] D. Moody, R. Perlner, A. Regenscheid, A. Robinson e D. Cooper, «Transition to Post-Quantum Cryptography Standards,» National Institute of Standards e Technology (NIST), NIST Internal Report (IR) 8547, nov. 2024, Initial Public Draft. Gaithersburg, MD, USA. indirizzo: <https://doi.org/10.6028/NIST.IR.8547.ipd>
- [5] S. Pakin e E. G. Rieffel, «Introduction to Quantum Computing,» in *Quantum Computing: A Gentle Introduction*, NASA, 2024.
- [6] S. Pichai. «Interview: The Roadmap for AI and Quantum Computing.» Il CEO di Google posiziona i computer quantistici "praticamente utili" in un orizzonte di 5-10 anni, paragonando la fase attuale ai primi giorni dell'AI. indirizzo: <https://blog.google/technology/research/>
- [7] Gartner Inc., «Hype Cycle for Compute, 2024,» Gartner, Research Report, lug. 2024, Classifica il Quantum Computing nella fase delle 'Aspettative Gonfiate' con un tempo di

- adozione mainstream stimato superiore ai 10 anni. indirizzo:
<https://www.gartner.com/en/newsroom/>
- [8] McKinsey & Company, «Quantum Technology Monitor 2024,» McKinsey & Company, rapp. tecn., apr. 2024, Proietta il pieno valore economico e la tolleranza ai guasti (fault-tolerance) verso il 2035. indirizzo:
<https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/quantum-technology-monitor>
- [9] EITCA. «Perché la sicurezza del sistema crittografico RSA dipende dalla difficoltà di fattorizzare numeri composti di grandi dimensioni e in che modo ciò influenza le dimensioni delle chiavi consigliate? »Indirizzo: <https://it.eitca.org/cybersecurity/eitc-is-ccf-classical-cryptography-fundamentals/introduction-to-public-key-cryptography/the-rsa-cryptosystem-and-efficient-exponentiation/examination-review-the-rsa-cryptosystem-and-efficient-exponentiation/why-is-the-security-of-the-rsa-cryptosystem-dependent-on-the-difficulty-of-factoring-large-composite-numbers-and-how-does-this-influence-the-recommended-key-sizes/>
- [10] J. Preskill e J. Eisert, «Mind the gaps: The fraught road to quantum advantage,» *arXiv:2510.19928v1*, ott. 2025. indirizzo:
<https://arxiv.org/html/2510.19928v1#bib>
- [11] B. Rampazzo, «Post-Quantum Anonymous Credentials and ZK-STARKs,» Master's Thesis. Available as provided., tesi di laurea mag., Politecnico di Torino University, 2023.
- [12] CISA, NSA, and NIST, *Quantum-Readiness: Migration to Post-Quantum Cryptography*, Cybersecurity Information Sheet (CSI), Documento congiunto delle agenzie federali USA che raccomanda l'approccio ibrido e definisce i rischi per i sistemi legacy., ago. 2023. indirizzo:
<https://www.cisa.gov/resources-tools/resources/quantum-readiness-migration-post-quantum-cryptography>
- [13] J. Preskill, «Quantum computing in the NISQ era and beyond,» *Quantum*, vol. 2, p. 79, 2018, Paper che definisce i limiti attuali dell'hardware.
- [14] J. Howe, T. Oder, M. Kraus e T. Güneysu, «Standard Lattices in Hardware,» *Design Automation Conference (DAC)*, 2018, Paper accademico molto citato nelle iterazioni NIST che analizza i colli di bottiglia energetici e computazionali nell'implementazione hardware PQC su dispositivi IoT limitati. indirizzo:
<https://dl.acm.org/doi/10.1145/2897937.2898037>

- [15] Entrust Cybersecurity Institute e Ponemon Institute, «2024 PKI and Post-Quantum Trends Study,» Entrust Corporation, Global Survey, ott. 2024, Rileva che il 59% delle organizzazioni considera la complessità dell'orchestrazione PKI su sistemi esistenti una sfida critica e che il 43% non ha visibilità completa sui propri asset crittografici (legacy). indirizzo: <https://www.entrust.com/resources/reports/2024-pki-and-post-quantum-trends-study>
- [16] W. Barker, W. Polk e M. Souppaya, «Migration to Post-Quantum Cryptography: Preparation for Considering the Implementation and Adoption of Quantum Safe Cryptography,» National Institute of Standards e Technology (NIST), NIST Special Publication (SP) 1800-38, 2023, Discute in dettaglio l'esigenza di Crypto-Agility e le sfide architetturali. indirizzo: [https://csrc.nist.gov/pubs/sp/1800/38/iprd-\(1\)](https://csrc.nist.gov/pubs/sp/1800/38/iprd-(1))
- [17] V. Smyslov, C. Tjhai, M. Shen e P. Kampanakis, «Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2),» Internet Engineering Task Force (IETF), Request for Comments 9370, mag. 2023, Esempio perfetto delle sfide di integrazione a livello architetturale: la standardizzazione necessaria per supportare scambi di chiavi ibridi classici/PQC. DOI: [10.17487/RFC9370](https://doi.org/10.17487/RFC9370) indirizzo: <https://datatracker.ietf.org/doc/rfc9370/>
- [18] G. Alagic et al., «Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process,» National Institute of Standards e Technology (NIST), NIST Interagency/Internal Report (NISTIR) 8413, 2022, Fondamentale per i benchmark su dispositivi limitati (Cortex-M4) e le dimensioni delle chiavi. DOI: [10.6028/NIST.IR.8413](https://doi.org/10.6028/NIST.IR.8413)
- [19] European Union Agency for Cybersecurity (ENISA), «Post-Quantum Cryptography: Integration study,» ENISA, rapp. tecn., mag. 2021, Tratta ampiamente l'impatto della PQC sui protocolli di rete (TLS, IKEv2), il problema della frammentazione dei pacchetti e i limiti dei dispositivi IoT. indirizzo: <https://www.enisa.europa.eu/publications/post-quantum-cryptography-integration-study>
- [20] Gartner, *Gartner Hype Cycle for Emerging Technologies*, <https://www.gartner.com/en/articles/hype-cycle-for-emerging-technologies>, Report annuale sulle tecnologie emergenti, 2024.
- [21] M. Dyakonov, «The case against quantum computing,» *IEEE Spectrum*, vol. 56.n. 3, pp. 24–57, 2019, Un punto di vista critico e scettico utile per bilanciare l'hype.
- [22] C. Gidney e M. Ekerå, «How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,» *Quantum*, vol. 5, p. 433, 2021, Dimostrazione matematica del perché non possiamo craccare RSA oggi.

- [23] M. Schuld e N. Killoran, «Is Quantum Advantage the Right Goal for Quantum Machine Learning?» *PRX Quantum*, vol. 3.n. 3, lug. 2022, issn: 2691-3399. doi: [10.1103/prxquantum.3.030101](https://doi.org/10.1103/prxquantum.3.030101) indirizzo: <http://dx.doi.org/10.1103/PRXQuantum.3.030101>

Appendice A

Codice Sorgente

A.1 5.2

```
1 <properties>
2   <bouncycastle.version>1.83</bouncycastle.version>
3 </properties>
4 <dependencies>
5   <dependency>
6     <groupId>org.bouncycastle</groupId>
7     <artifactId>bcprov-jdk18on</artifactId>
8     <version>${bouncycastle.version}</version>
9   </dependency>
10  <dependency>
11    <groupId>org.bouncycastle</groupId>
12    <artifactId>bcpkix-jdk18on</artifactId>
13    <version>${bouncycastle.version}</version>
14  </dependency>
15  <dependency>
16    <groupId>org.bouncycastle</groupId>
17    <artifactId>bcutil-jdk18on</artifactId>
18    <version>${bouncycastle.version}</version>
19  </dependency>
20 </dependencies>
```

Listing A.1: Codice XML relativo all'integrazione delle dipendenze Bouncycastle all'interno del file di dipendenze pom.xml

A.2 5.3

```
1 @Service
2 public class HybridCryptoService {
3
```

```

4  private KeyPair serverPqcKeyPair;
5  private KeyPair serverEccKeyPair;
6  private static final String KEY_GENERATION_SEED = "
    QUESTA_FRASE_GENERA_SEMPRE_LE_STESSE_CHIAVI_PER_ASTROVERSE_2025";
7
8  static {
9      Security.addProvider(new BouncyCastleProvider());
10 }
11
12 // Al'avvio del server, generiamo le chiavi MASTER del server.
13 @PostConstruct
14 public void init() throws Exception {
15     System.out.println("Generazione Chiavi Deterministiche in corso..."
16         );
17
18     // Creiamo un generatore di casualita' che parte sempre dallo
19     // stesso punto
20     SecureRandom deterministicRandom = SecureRandom.getInstance("
21         SHA1PRNG");
22     deterministicRandom.setSeed(KEY_GENERATION_SEED.getBytes());
23
24     // 1. Genera ML-KEM (Kyber) usando il random deterministico
25     KeyPairGenerator pqcKpg = KeyPairGenerator.getInstance("ML-KEM", "
26         BC");
27     pqcKpg.initialize(MLKEMParameterSpec.ml_kem_768,
28         deterministicRandom);
29     serverPqcKeyPair = pqcKpg.generateKeyPair();
30
31     // 2. Genera ECC usando lo STESSO random (che ora è avanzato di
32     // stato)
33     KeyPairGenerator ecKpg = KeyPairGenerator.getInstance("ECDH", "BC")
34         ;
35     ecKpg.initialize(new ECGenParameterSpec("secp256r1"),
36         deterministicRandom);
37     serverEccKeyPair = ecKpg.generateKeyPair();
38
39     System.out.println("Chiavi Server pronte (Stabili tra i riavvii)");
40 }

```

Listing A.2: HybridCryptoService.java inizializzazione BouncyCastle e seed

A.3 5.3.1

```

1  public String hashString(String input) {
2      try {
3          MessageDigest digest = MessageDigest.getInstance("SHA-256");

```

```
4         byte[] hash = digest.digest(input.getBytes(StandardCharsets.  
5             UTF_8));  
6         return Base64.getEncoder().encodeToString(hash);  
7     } catch (Exception e) {  
8         throw new RuntimeException("Hashing fallito", e);  
9     }  
}
```

Listing A.3: HybridCryptoService.java metodo per cifrare il testo in chiaro in modo da generare un indice

A.4 5.3.1

```
1     private SecretKey combineSecrets(byte[] s1, byte[] s2) throws Exception  
2     {  
3         MessageDigest d = MessageDigest.getInstance("SHA-256");  
4         d.update(s1);  
5         d.update(s2);  
6         return new SecretKeySpec(d.digest(), "AES");  
7     }
```

Listing A.4: HybridCryptoService.java metodo che combina le chiavi segrete delle due componenti in un unico segreto condiviso

A.5 5.3.2

```
1     private String aesEncrypt(String data, SecretKey key) throws Exception  
2     {  
3         javax.crypto.Cipher c = javax.crypto.Cipher.getInstance("AES/GCM/  
4             NoPadding", "BC");  
5         byte[] iv = new byte[12];  
6         new SecureRandom().nextBytes(iv);  
7         c.init(javax.crypto.Cipher.ENCRYPT_MODE, key, new javax.crypto.spec  
8             .GCMParameterSpec(128, iv));  
9         byte[] ct = c.doFinal(data.getBytes());  
10        byte[] out = new byte[iv.length + ct.length];  
11        System.arraycopy(iv, 0, out, 0, iv.length);  
12        System.arraycopy(ct, 0, out, iv.length, ct.length);  
13        return Base64.getEncoder().encodeToString(out);  
14    }
```

Listing A.5: HybridCryptoService.java metodo per cifrare la chiave finale

A.6 5.3.2

```

1  private String aesDecrypt(String data, SecretKey key) throws Exception
2  {
3      byte[] in = Base64.getDecoder().decode(data);
4      byte[] iv = new byte[12];
5      System.arraycopy(in, 0, iv, 0, 12);
6      javax.crypto.Cipher c = javax.crypto.Cipher.getInstance("AES/GCM/
7      NoPadding", "BC");
8      c.init(javax.crypto.Cipher.DECRYPT_MODE, key, new javax.crypto.spec
9      .GCMParameterSpec(128, iv));
10     return new String(c.doFinal(in, 12, in.length - 12));
11 }

```

Listing A.6: HybridCryptoService.java metodo per decifrare la chiave finale

A.7 5.3.2

```

1  // Record contenitore per i risultati
2  public record EncryptionResult(String encryptedData, String
3      encapsulation, String clientEccPub) {}
4
5  /**
6   * FASE 1: CIFRATURA
7   * Crea le chiavi e cifra il dato.
8   */
9  public EncryptionResult encrypt(String plainData) {
10     try {
11         // 1. Simula Client ECC
12         KeyPairGenerator clientEcKpg = KeyPairGenerator.getInstance("
13         ECDH", "BC");
14         clientEcKpg.initialize(new ECGenParameterSpec("secp256r1"));
15         KeyPair clientEcPair = clientEcKpg.generateKeyPair();
16
17         // 2. ECDH Agreement
18         KeyAgreement ecAgreement = KeyAgreement.getInstance("ECDH", "BC
19         ");
20         ecAgreement.init(clientEcPair.getPrivate());
21         ecAgreement.doPhase(serverEccKeyPair.getPublic(), true);
22         byte[] classicalSecret = ecAgreement.generateSecret();
23
24         // 3. ML-KEM Encapsulation
25         KeyGenerator kemGen = KeyGenerator.getInstance("ML-KEM", "BC");
26         kemGen.init(new KEMGenerateSpec(serverPqcKeyPair.getPublic(), "
27         Generic"));

```

```

24         SecretKeyWithEncapsulation kemResult = (
25             SecretKeyWithEncapsulation) kemGen.generateKey();
26
27         // 4. Chiave Finale
28         SecretKey finalKey = combineSecrets(classicalSecret, kemResult.
29             getEncoded());
30
31         // 5. AES Encrypt
32         String cipherText = aesEncrypt(plainData, finalKey);
33
34         return new EncryptionResult(
35             cipherText,
36             Base64.getEncoder().encodeToString(kemResult.getEncapsulation()
37                 ),
38             Base64.getEncoder().encodeToString(clientEcPair.getPublic().
39                 getEncoded())
40         );
41     } catch (Exception e) {
42         throw new RuntimeException("Errore cifratura Quantum", e);
43     }
44 }

```

Listing A.7: HybridCryptoService.java metodo per il processo di cifratura ibrida

A.8 5.3.3

```

1  /**
2   * FASE 2: DECIFRATURA
3   * Il server usa i metadati salvati nel DB per ricostruire la chiave.
4   */
5  public String decrypt(String encryptedData, String encapsulationStr,
6      String clientEccPubStr) {
7      try {
8          // Ricostruisce chiave pubblica client
9          byte[] clientPubBytes = Base64.getDecoder().decode(
10              clientEccPubStr);
11          KeyFactory kf = KeyFactory.getInstance("ECDH", "BC");
12          PublicKey clientEccPub = kf.generatePublic(new
13              X509EncodedKeySpec(clientPubBytes));
14
15          // ECDH
16          KeyAgreement ecAgreement = KeyAgreement.getInstance("ECDH", "BC");
17          ecAgreement.init(serverEccKeyPair.getPrivate());
18          ecAgreement.doPhase(clientEccPub, true);
19          byte[] classicalSecret = ecAgreement.generateSecret();

```

```

17
18      // ML-KEM Decapsulate
19      byte[] encapsulation = Base64.getDecoder().decode(
20          encapsulationStr);
21      KeyGenerator kemGen = KeyGenerator.getInstance("ML-KEM", "BC");
22      kemGen.init(new KEMExtractSpec(serverPqcKeyPair.getPrivate(),
23          encapsulation, "Generic"));
24      SecretKeyWithEncapsulation kemResult = (
25          SecretKeyWithEncapsulation) kemGen.generateKey();
26
27      // Chiave Finale
28      SecretKey finalKey = combineSecrets(classicalSecret, kemResult.
29          getEncoded());
30
31      return aesDecrypt(encryptedData, finalKey);
32  } catch (Exception e) {
33      throw new RuntimeException("Errore decifratura Quantum", e);
34  }
35  }

```

Listing A.8: HybridCryptoService.java metodo per il processo di decifratura ibrida

A.9 5.4

```

1  private final HybridCryptoService hybridCryptoService;
2
3  public UserService(UserRepository userRepository, UserSpaceRepository
4      userSpaceRepository, HybridCryptoService hybridCryptoService) {
5      this.userSpaceRepository = userSpaceRepository;
6      this.userRepository = userRepository;
7      this.hybridCryptoService = hybridCryptoService;
8  }
9
10 public User saveUser(User user) {
11     if (userRepository.existsByEmail(user.getEmail())) {
12         throw new IllegalArgumentException("Email già in uso");
13     }
14     // Controllo duplicati su hash
15     String emailHash = hybridCryptoService.hashString(user.getEmail());
16     if (userRepository.existsByEmailHash(emailHash)) {
17         throw new IllegalArgumentException("Email già in uso");
18     }
19     if (userRepository.existsByUsername(user.getUsername())) {
20         throw new IllegalArgumentException("Username già in uso");
21     }
22     // Eseguiamo la cifratura Ibrida Post-Quantum

```



```
22     var encryptionResult = hybridCryptoService.encrypt(user.getEmail())
23     ;
24     // Impostiamo i dati nel model User
25     user.setEmail(encryptionResult.encryptedData()); // Salviamo il
        cifrato
26     user.setEmailHash(emailHash); // Salviamo l'hash per le ricerche
27     user.setQuantumEncapsulation(encryptionResult.encapsulation());
28     user.setClientEccPublicKey(encryptionResult.clientEccPub());
29     user.setQuantumEncrypted(true);
30     return userRepository.save(user);
31 }
32
33 public User getUser(String email) {
34     // Effettuiamo la ricerca per hash poiché non possiamo farla per
        email in chiaro
35     String emailHash = hybridCryptoService.hashString(email);
36     User user = userRepository.findByEmailHash(emailHash).orElse(null);
37     if (user == null) {
38         throw new IllegalArgumentException("L'utente non esiste");
39     }
40     // Se l'utente è cifrato, decifriamo l'email prima di restituire l'
        oggetto
41     // così il controller riceve l'email leggibile
42     if (user.isQuantumEncrypted()) {
43         String decryptedEmail = hybridCryptoService.decrypt(
44             user.getEmail(),
45             user.getQuantumEncapsulation(),
46             user.getClientEccPublicKey()
47         );
48         user.setEmail(decryptedEmail); // Sostituiamo temporaneamente
        nell'oggetto
49     }
50     return user;
51 }
52
53 public User changeUserData(long id, String email, String username,
        String name, String lastName, String oldEmail, String oldUsername) {
54     // Calcoliamo il nuovo hash
55     String newEmailHash = hybridCryptoService.hashString(email);
56     if (userRepository.existsByEmailHashAndIdNot(newEmailHash, id) && !
        email.equals(oldEmail)) {
57         throw new IllegalArgumentException("Email già in uso");
58     } else if (userRepository.existsByUsernameAndIdNot(username, id) &&
        !username.equals(oldUsername)) {
59         throw new IllegalArgumentException("Username già in uso");
60     }
```

```
61 // Recuperiamo l'utente
62 User user = userRepository.getUserById(id);
63 if (user == null) {
64     throw new RuntimeException("Utente non trovato");
65 }
66 // Se l'email è cambiata, dobbiamo ricifrare tutto
67 if (!email.equals(oldEmail)) {
68     var encryptionResult = hybridCryptoService.encrypt(email);
69     user.setEmail(encryptionResult.encryptedData());
70     user.setEmailHash(newEmailHash);
71     user.setQuantumEncapsulation(encryptionResult.encapsulation());
72     user.setClientEccPublicKey(encryptionResult.clientEccPub());
73     user.setQuantumEncrypted(true);
74 }
75 // Aggiorniamo gli altri campi
76 user.setUsername(username);
77 user.setName(name);
78 user.setCognome(lastName);
79 return userRepository.save(user);
80 }
81
82 public User getUserData(Long id) {
83     if (!userRepository.existsById(id)) {
84         throw new IllegalArgumentException("Utente non esistente");
85     }
86     // Per vedere la mail in chiaro nell'account
87     User user = userRepository.getUserById(id);
88     if (user.isQuantumEncrypted()) {
89         user.setEmail(hybridCryptoService.decrypt(
90             user.getEmail(), user.getQuantumEncapsulation(), user.
91                 getClientEccPublicKey()
92         ));
93     }
94     return user;
95 }
```

Listing A.9: Snippet di metodi di UserService.java

A.10 5.4

```
1 @JsonProperty("isQuantumEncrypted")
2 private boolean isQuantumEncrypted;
```

Listing A.10: Codice aggiunto a UserDTO.java

A.11 5.4.1

```
1  @Lob    // il testo cifrato è lungo
2  @Column(columnDefinition = "LONGTEXT")
3  private String email;
4  @Column(unique = true)
5  private String emailHash;
6  private boolean isQuantumEncrypted = false;
7  @Lob // Contenitore per la chiave
8  @Column(columnDefinition = "LONGTEXT")
9  private String quantumEncapsulation;
10 @Lob // Chiave pubblica effimera del client
11 @Column(columnDefinition = "LONGTEXT")
12 private String clientEccPublicKey;
```

Listing A.11: Codice Java relativo alla modifica dell'Entity User

A.12 5.4.1

```
1  boolean existsByEmailHash(String emailHash);
2  // Per il metodo changeUserData
3  boolean existsByEmailHashAndIdNot(String emailHash, Long id);
4  Optional<User> findByEmailHash(String emailHash);
5  // Per la migrazione PQC
6  List<User> findByIsQuantumEncryptedFalse();
```

Listing A.12: Codice Java relativo alla UserRepository

A.13 5.4.2

```
1
2 @RestController
3 @RequestMapping("/api/auth")
4 public class AuthController {
5     private final JwtUtil jwtUtil;
6     private final UserService userService;
7     private final TokenBlackListService tokenBlackListService;
8     public AuthController(JwtUtil jwtUtil, UserService userService,
9         TokenBlackListService tokenBlackListService) {
10         this.jwtUtil = jwtUtil;
11         this.userService = userService;
12         this.tokenBlackListService = tokenBlackListService;
13     }
14 }
```

```
13
14 @PostMapping("/registration")
15 public ResponseEntity<?> registrationUser(@RequestParam String nome,
16     @RequestParam String cognome, @RequestParam String email,
17     @RequestParam String username, @RequestParam String password) {
18     Map<String, String> response = new HashMap<>();
19     String originalEmail = email;
20     User user = new User(nome, cognome, username, email, password);
21     if (!isValidText(user.getNome(), namesRegex)) {
22         response.put("error", "Nome non valido");
23         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).
24             body(response);
25     }
26     if (!isValidText(user.getCognome(), namesRegex)) {
27         response.put("error", "Cognome non valido");
28         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).
29             body(response);
30     }
31     if (!isValidText(user.getUsername(), usernameRegex)) {
32         response.put("error", "Username non valido");
33         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
34             response);
35     }
36     if (!isValidText(user.getEmail(), emailRegex)) {
37         response.put("error", "Email non valida");
38         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
39             response);
40     }
41     if (!isValidText(user.getPassword(), passwordRegex)) {
42         response.put("error", "Password non valida");
43         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
44             response);
45     }
46     String hashPassword = Hash.hashPassword(user.getPassword());
47     user.setPassword(hashPassword);
48     try {
49         User userWithId = userService.saveUser(user);
50         String accessToken = jwtUtil.generateToken(
51             userWithId.getId(),
52             originalEmail,
53             //Qui è importante utilizzare l'email in chiaro affinché
54             //é venga generato correttamente il token
55             //Con getEmail altrimenti, poiché è già stata eseguita
56             //saveUser si avrebbe l'email cifrata nel token
57             user.getUsername(),
58             user.getNome(),
59             user.getCognome(),
```

```
51         user.isAdmin());
52     if (accessToken == null) {
53         response.put("error", "Errore nella generazione del token");
54         ;
55         return ResponseEntity.status(HttpStatus.
56             INTERNAL_SERVER_ERROR).body(response);
57     }
58     response.put("accessToken", accessToken);
59     return ResponseEntity.ok(response);
60 } catch (IllegalArgumentException e) {
61     response.put("error", "Utente già esistente");
62     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
63         response);
64 }
65 }
66
67 @PostMapping("/login")
68 public ResponseEntity<?> loginUser(@RequestParam String email,
69     @RequestParam String password) {
70     Map<String, String> response = new HashMap<>();
71     try {
72         User user = userService.getUser(email);
73         String userPassword = user.getPassword();
74         if(!Hash.checkPassword(password, userPassword)) {
75             response.put("error", "Password errata");
76             return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
77                 response);
78         }
79         String accessToken = jwtUtil.generateToken(
80             user.getId(),
81             user.getEmail(), //Qui getEmail è leggibile perché è
82                 stata decifrata da getUser (vedi UserService)
83             user.getUsername(),
84             user.getNome(),
85             user.getCognome(),
86             user.isAdmin()
87         );
88         response.put("accessToken", accessToken);
89         return ResponseEntity.ok(response);
90     } catch (IllegalArgumentException e) {
91         response.put("error", "L'utente non esiste");
92         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
93             response);
94     }
95 }
96
97 @PostMapping("/change-user-data")
```

```
91 public ResponseEntity<?> changeUserData(@RequestParam String nome,
92   @RequestParam String cognome, @RequestParam String email,
93   @RequestParam String username, @RequestParam String psw,
94   @RequestParam String confirmPassword, @RequestParam String
95   vecchiaPassword, @RequestHeader("Authorization") String token) {
96   Map<String, String> response = new HashMap<>();
97   String nuovaEmailInChiaro = email; // Inizializziamo con la mail in
98   chiaro che andremo ad utilizzare dopo
99   // poiché nelle successive operazioni, mail andrà hashata
100   User user = new User(nome, cognome, username, email, psw);
101   ChangeUserRequest request = new ChangeUserRequest(user,
102     confirmPassword, vecchiaPassword);
103   try {
104     token = token.replace("Bearer ", "");
105     String confermaPassword = request.getConfermaPassword();
106     String password = user.getPassword();
107     DecodedJWT decodedJWT = JwtUtil.JwtDecode(token);
108     long id = decodedJWT.getClaim("id").asLong();
109     String oldEmail = decodedJWT.getClaim("email").asString();
110     String oldUsername = decodedJWT.getClaim("username").asString()
111     ;
112     if (user.getEmail().isEmpty() || user.getUsername().isEmpty()
113         || user.getNome().isEmpty() || user.getCognome().isEmpty())
114     {
115       response.put("error", "Uno o più dati dell'utente mancanti"
116         );
117       return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
118         response);
119     } else if (!isValidText(user.getEmail(), emailRegex)) {
120       response.put("error", "Email non valida");
121       return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
122         response);
123     } else if (!isValidText(user.getNome(), namesRegex)) {
124       response.put("error", "Nome non valido");
125       return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
126         response);
127     } else if (!isValidText(user.getCognome(), namesRegex)) {
128       response.put("error", "Cognome non valido");
129       return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
130         response);
131     } else if (!isValidText(user.getUsername(), usernameRegex)) {
132       response.put("error", "Username non valido");
133       return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
134         response);
135     } else {
136       user = userService.changeUserData(id, user.getEmail(), user
137         .getUsername(), user.getNome(), user.getCognome(),
```

```
        oldEmail, oldUsername);
    }
    if(!password.isEmpty()) {
        String oldPassword = userService.getOldPassword(id);
        if(!isValidText(password, passwordRegex)) {
            response.put("error", "Password non valida");
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).
                body(response);
        } else if(!password.equals(confermaPassword)) {
            response.put("error", "La password di conferma non è
                diversa dalla nuova password");
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).
                body(response);
        } else if (!Hash.checkPassword(request.getVecchiaPassword(),
            oldPassword)) {
            response.put("error", "La password corrente non è
                corretta");
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).
                body(response);
        } else {
            String hashPassword = Hash.hashPassword(password);
            user.setPassword(hashPassword);
            userService.changePassword(user.getUsername(),
                hashPassword);
        }
    }
    String newToken = jwtUtil.generateToken(id, nuovaEmailInChiaro,
        user.getUsername(), user.getNome(), user.getCognome(), user
        .isAdmin());
    TokenBlackList tokenBlackList = new TokenBlackList();
    tokenBlackList.setAccessToken(token);
    tokenBlackListService.saveAccessToken(tokenBlackList);
    response.put("message", "Dati aggiornati");
    response.put("accessToken", newToken);
    return ResponseEntity.ok(response);
} catch (RuntimeException e) {
    response.put("error", e.getMessage());
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
        response);
}
}

@GetMapping("/view-account")
public ResponseEntity<?> viewAccount(@RequestHeader("Authorization")
    String token) {
    Map<String, Object> response = new HashMap<>();
    try {
```

```
157     token = token.replace("Bearer ", "");
158     if(!jwtUtil.isTokenValid(token)) {
159         response.put("error", "Token non valido");
160         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(
161             response);
162     }
163     DecodedJWT decodedJWT = JwtUtil.JwtDecode(token);
164     Long idString = decodedJWT.getClaim("id").asLong();
165     if (idString == null) {
166         response.put("error", "Id utente non presente");
167         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
168             response);
169     }
170     User user = userService.getUserData(idString);
171     UserDTO userDTO = new UserDTO(user.getUserSpaces(), user.
172         getPosts(), user.getNome(), user.getCognome(), user.
173         getUsername(), user.getEmail(), user.isQuantumEncrypted());
174     response.put("user", userDTO);
175     response.put("spaces", userService.getSpaceByUser(user));
176     return ResponseEntity.ok(response);
177 } catch (IllegalArgumentException e) {
178     response.put("error", "L'utente non esiste");
179     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
180         response);
181 }
```

Listing A.13: Snippet di metodi di AuthController.java

A.14 5.4.3

```
1 @Component
2 public class QuantumMigration {
3
4     private final UserRepository userRepository;
5     private final HybridCryptoService cryptoService;
6
7     public QuantumMigration(UserRepository userRepository,
8         HybridCryptoService cryptoService) {
9         this.userRepository = userRepository;
10        this.cryptoService = cryptoService;
11    }
12
13    @EventListener(ApplicationReadyEvent.class)
14    public void migrateLegacyUsers() {
```



```
14 // Cerchiamo utenti che hanno il flag isQuantumEncrypted =
    FALSE (0)
15 List<User> legacyUsers = userRepository.
    findByIsQuantumEncryptedFalse();
16 if (legacyUsers.isEmpty()) {
17     return; // Nessuno da migrare
18 }
19 System.out.println(" [MIGRAZIONE] Trovati " + legacyUsers.size
    () + " utenti vulnerabili. Avvio procedura Post-Quantum...")
    ;
20 for (User user : legacyUsers) {
21     try {
22         String emailInChiaro = user.getEmail();
23         // Se per qualche motivo l'email è nulla o vuota,
            saltiamo
24         if (emailInChiaro == null || emailInChiaro.isEmpty())
            continue;
25         System.out.println("    -> Migrazione utente: " + user.
            getUsername() + " (" + emailInChiaro + ")");
26         // CIFRATURA (Protezione Privacy)
27         // Usiamo Kyber + AES per rendere l'email illeggibile
28         var encryptionResult = cryptoService.encrypt(
            emailInChiaro);
29         // HASHING (Indicizzazione)
30         // Creiamo l'hash SHA-256 per permettere il login
            futuro
31         String emailHash = cryptoService.hashString(
            emailInChiaro);
32         // AGGIORNAMENTO DEL RECORD
33         user.setEmail(encryptionResult.encryptedData()); //
            Sovrascriviamo il chiaro col cifrato
34         user.setEmailHash(emailHash);
35         // Salviamo le chiavi pubbliche per poter decifrare
            dopo
36         user.setQuantumEncapsulation(encryptionResult.
            encapsulation());
37         user.setClientEccPublicKey(encryptionResult.
            clientEccPub());
38         // Settiamo il flag a TRUE: ora l'utente è al sicuro
39         user.setQuantumEncrypted(true);
40         userRepository.save(user);
41         System.out.println(" Successo! Utente protetto contro
            attacchi quantistici.");
42     } catch (Exception e) {
43         System.err.println(" Errore durante la migrazione: " +
            e.getMessage());
44     }
```

```
45         }  
46     }  
47 }
```

Listing A.14: Componente di migrazione QuantumMigration.java

A.15 5.5

```
1  onMounted(async () => {  
2      // Recupero dati utente dal backend  
3      await apiURLToken.get('/auth/view-account')  
4      .then((response) => {  
5          user.value = response.data.user;  
6          // --- VERIFICA STATO PQC ---  
7          // 1. Controlla se il backend conferma la cifratura (flag dal DB)  
8          // 2. Controlla se la notifica e' gia' stata mostrata in questa  
9              sessione  
10         if (user.value.isQuantumEncrypted && !sessionStorage.getItem('quantum_toast_shown')) {  
11             // Feedback visivo all'utente  
12             toast.success("Accesso sicuro: Crittografia Post-Quantum attiva!"  
13                 , {  
14                     duration: 5000,  
15                     dismissible: true  
16                 });  
17             // Flag di sessione per non ripetere la notifica  
18             sessionStorage.setItem('quantum_toast_shown', 'true');  
19         } else if (!user.value.isQuantumEncrypted && !sessionStorage.  
20             getItem('quantum_toast_shown')) {  
21             toast.warning("Non sei protetto contro gli attacchi post-quantum"  
22                 )  
23         }  
24     })  
25     .catch((error) => {  
26         console.error(error);  
27     });  
28 }
```

Listing A.15: Logica Vue.js per la notifica di sicurezza Post-Quantum